Institutional Capability Lineage Architecture

Registry-Centered Governance of Capability Evolution in AI-Enabled Organizations

Abstract. *Enterprise capability and conceptual models represent organizational purpose, context, realization, and change, but AI-enabled execution creates a continuity problem: recurring capabilities must remain accountable as human–machine arrangements and infrastructures change. This paper introduces the* Institutional Capability Lineage Architecture *(ICLA), a registry-centered conceptual reference architecture connecting stable capability identity to versioned knowledge-and-evaluation contracts, contextual execution, evidence, and governed change. Governed knowledge remains distributed, while immutable* Capability Knowledge Contracts *(CKCs) specify the commitments for each execution. ICLA models incremental institutional evolution along two dimensions: approved succession appends a complete CKC and decision-linked delta under an existing capability identity; governed crystallization assigns a new identity and initial CKC to a recurrent responsibility that passes review. Both preserve records needed to interpret earlier executions. Following design-science research, we iteratively refine the artifact through a purposive, responsibility-oriented literature synthesis, explicit requirements, and boundary alternatives, then specify a conceptual meta-model and proposed conformance invariants. Constructed OAuth 2.1 and capability-formation traces demonstrate the architecture. The meta-model and invariants assess structural consistency; linked records and schemas assess representational consistency; and deterministic tests assess executable consistency through successor activation and governed capability formation. ICLA extends enterprise capability and conceptual modeling with complementary principles from runtime models, model evolution, provenance, process discovery, and organizational memory. The summative artificial evaluation supports architectural coherence and technical feasibility for the exercised succession and submitted-proposal formation paths; discovery effectiveness and organizational effectiveness remain prospective.*

Keywords. conceptual modeling • enterprise modeling • institutional capability lineage architecture • capability crystallization • capability meta-models • reference architecture • AI-enabled organizations

1 Introduction

Conceptual and enterprise modeling make organizational and information-system phenomena explicit for communication, analysis, and design (Frank 2014). Capability-driven development connects an enterprise's intended abilities to goals, context, performance, processes, and delivery, while capability meta-models represent observation, decision, adjustment, and change (Koutsopoulos et al. 2021; Stirna et al. 2012; Zdravkovic et al. 2013). Complementary traditions keep models operational at runtime, relate heterogeneous models, and preserve controlled evolution (Blair et al. 2009; Fill 2018; Redlich et al. 2014; Salay et al. 2020; Stojanovic et al. 2003). Together, they establish that capability, context, runtime state, model relations, and change can be modeled explicitly.

Execution supported by artificial intelligence (AI) adds a continuity question at the boundary between these models and operational work. Contemporary AI execution provides mechanisms for retrieval, agent memory, context assembly, skills, and tool use (Gao et al. 2026; Lewis et al. 2020; Mei et al. 2025; Z. Zhang et al. 2025); ICLA allows these mechanisms to be combined within one capability instance. Multi-agent and human–agent research describes arrangements in which people and agents occupy differentiated interaction and coordination roles (Wu et al. 2024; Zou

et al. 2026); an institutional execution boundary may additionally include workflows, services, validators, and accountable approvers. AI intensifies an existing separation: reconfigurable human–machine arrangements may change their memory, tools, coordination structures, and degree of local autonomy while the institutional responsibility persists.

Organizations may replace platforms and tools while retaining capabilities such as credit-risk assessment, incident diagnosis, or compliance validation. Organizational-memory and knowledge-management research explains how organizational knowledge is retained and retrieved across individuals, routines, structures, artifacts, and information systems (Alavi and Leidner 2001; Stein 1995; Walsh and Ungson 1991). Taken together, these traditions do not, however, make the complete governed trace their central object. That trace connects a stable capability identity to the exact knowledge-and-evaluation commitment used in a situated execution, the resulting evidence, and any authorized successor state.

Consider an organization migrating several services to OAuth 2.1. Security architects interpret policy, coding agents modify individual services, validation systems collect compatibility evidence, and accountable reviewers authorize releases. Execution may produce both a service-local migration exception and a reusable compatibility-test pattern relevant to later migrations. Both may be retained in agent state, reports, repositories, or decision records, but those locations do not determine whether either result remains local or becomes authoritative for future work. The organization therefore needs a stable capability address, an exact record of the knowledge-and-evaluation commitments governing each execution, and a governed path that distinguishes local evidence from authorized changes to later capability contracts without rewriting earlier history.

Distributed execution requires a canonical capability contract and a governed return path.

Reasoning and sources remain distributed; capability responsibility, authority, and change become explicit.

This paper proposes the *Institutional Capability Lineage Architecture* (ICLA), an implementation-agnostic conceptual reference architecture for AI-enabled execution regimes in which people, agents, tools, memories, and coordination structures may change while institutional responsibility persists. ICLA builds on models of capability, context, runtime state, and change by making the governed connection among persistent responsibility, situated execution, evidence, and authorized change an explicit unit of accountability. An *institutional capability* is a recurring, outcome-oriented responsibility with a stable identity, accountable owner, and governed lifecycle across implementations. The Registry gives it a stable address and records its identity, relations, lifecycle, and active *Capability Knowledge Contract* (CKC), while governed knowledge remains distributed.

We use *knowledge-for-capability* for the concepts, policies, methods, precedents, and evidence maintained to perform a recurring capability responsibly. Memory research distinguishes semantic from episodic memory and declarative from nondeclarative memory (Squire 1992; Tulving 1972). Drawing on these distinctions, ICLA treats these knowledge elements as serving potentially overlapping semantic, procedural, or episodic roles; these roles are analytical categories rather than prescribed storage layers. The architectural question is which commitments belong to a capability, who authorizes them, how an execution view is derived, and what evidence may justify changes to later versions.

A CKC is the consumer-independent contract for knowledge-for-capability:

A Capability Knowledge Contract is a governed, versioned, and consumer-independent specification of the knowledge scope, operational relations, obligations, authorities, evidence requirements, evaluation contract,

source bindings, and projection rules required to perform a recurring institutional capability under declared conditions.

Operational intents combine semantic content—the requested goal and problem context—with conditions for situated execution and assurance. They resolve against active CKCs and produce contextual assemblies for *Capability Execution Environments* (CEEs), situated boundaries for admitted capability instances that may be instantiated by people, teams, agent configurations, workflows, services, or hybrids. The assembly records a bounded mandate; the CEE controls local execution logic or reasoning and state; and selected outputs return under the evidence contract.

Governance may activate a successor CKC or separately review retained evidence of a recurrent responsibility for capability formation. These transitions evolve the institutional model incrementally along two dimensions: succession refines the commitments attached to a recognized responsibility, while crystallization extends the set of recognized responsibilities. The *institutional capability lineage*—not the Registry or CKC alone—is the paper's central relational contribution for this execution regime. It connects stable identity and authority to changing CEEs, exact contracts, assemblies, executions, evidence, decisions, and lifecycle changes within a single auditable continuity. It does not treat operational autonomy as institutional authority.

In this paper, ICLA denotes the reference architecture; an *institutional capability lineage* is its modeled authority-preserving relational structure; and a *lineage trace* is a concrete retained instantiation for one capability and its executions.

The paper addresses the following question:

How can a registry-centered architecture preserve institutional capability continuity while supporting canonical capability contracts, contextual execution, comparable evidence, and incremental governed evolution of both commitments and recognized responsibilities across heterogeneous execution environments?

ICLA is structured around four linked separations: capability identity from authorized CKC versions; a CKC from its execution-specific contextual assembly; governed commitments from CEE-local execution logic or reasoning; and submitted evidence from governed canonical change. Together, they prevent changes in execution, reasoning, or observed evidence from implicitly redefining institutional authority. These separations provide the structural basis for three contributions:

- *An authority-preserving capability-lineage meta-model and CKC abstraction* connecting stable institutional capability identity, immutable authorized contracts, contextual execution, evidence, and governed change across heterogeneous CEEs.
- *A two-dimensional incremental evolution model* in which decision-linked CKC succession refines an existing capability and governed crystallization forms a new capability from a recurrent responsibility supported by retained history, while both preserve earlier references.
- *A conformance and evaluation framework* comprising invariants, profiles, logical interfaces, machine-checkable schemas, constructed reference traces, deterministic tests, and falsifiable comparative claims.

We develop ICLA through design-science research. An initial artifact was iteratively refined through responsibility-oriented literature analysis, explicit requirements, boundary alternatives, and trace construction. An OAuth 2.1 reference trace demonstrates situated execution and CKC succession, while a complementary capability-formation trace exercises governed promotion from a submitted proposal to a new capability and initial CKC with separate activation. Linked records and schemas assess representational consistency, and deterministic tests assess executable consistency for the exercised paths. Discovery effectiveness and organizational effectiveness remain prospective. Sect. 2 positions the contribution; Sects. 3 and 4 report the method and derive the requirements; and Sects. 5–7 present, respectively, the

conceptual model, situated execution, and governed evolution. Sects. 8–11 define conformance, map implementation, demonstrate the artifact, and evaluate the current evidence; Sect. 12 examines the contribution and its boundaries before the conclusion.

2 Foundations and Related Work

ICLA is positioned primarily as an extension of enterprise capability and conceptual modeling. Runtime models, model evolution, model management, process discovery, provenance, and organizational memory provide complementary principles and comparison points, while AI literature characterizes the heterogeneous, distributed, and reconfigurable execution setting rather than serving as an equally central conceptual foundation. The comparison distinguishes responsibilities inherited from prior work from the authority-preserving capability-lineage relations introduced by ICLA.

2.1 Conceptual and multi-perspective enterprise modeling

Conceptual models provide abstractions for communication, analysis, and information-systems design. Enterprise models combine organizational and information-system perspectives and may integrate domain-specific languages through a common meta-modeling architecture (Frank 2014). This matters to ICLA because a recurring capability is simultaneously an organizational responsibility, knowledge boundary, evaluation object, and operational concern. ICLA makes these perspectives and their typed relations explicit rather than leaving them in implementation-specific prompts, repositories, or workflows.

Enterprise-model evolution also motivates separation between a stable modeling language and context-specific extensions. Semantic annotation, for example, can loosely couple formal schemata to enterprise models so that their representation and analysis scope can change without modifying the original modeling language (Fill 2018). ICLA applies a related separation between a consumer-independent CKC and its contextual assemblies. The analogy is limited: a CKC also specifies authority, evidence, evaluation, and activation rules, and is therefore not merely a semantic annotation.

Reference-architecture research has also shown how layered, model-driven architectures can connect domain models to configurable collaborative information systems (Hernandez-Mendez et al. 2018). ICLA shares the use of implementation-independent layers and interfaces, but its artifact is a governance meta-model for capability continuity rather than an architecture for configuring one class of collaborative application.

2.2 Capability-driven development and capability change

Capability-driven development (CDD) integrates organizational and information-system development by representing goals, key performance indicators (KPIs), capabilities, context, and capability-delivery patterns (Stirna et al. 2012). Subsequent work connects capability models to context-dependent service delivery and runtime adjustment (Zdravkovic et al. 2013). These contributions establish that a capability can be modeled as an intentional and measurable ability whose realization varies with context.

Research comparing capability meta-models shows that capability change is already represented through concepts for observation, measurement, context, intention, decision criteria, alternatives, realization, adjustment, and capability architecture (Koutsopoulos et al. 2021). Subsequent application and stakeholder evaluation of KYKLOS and Compass further demonstrate modeling support for changing capabilities and the trade-off between expressive detail and ease of use (Koutsopoulos et al. 2024). ICLA builds on contextual capability delivery, goals, KPIs, capability relations, and adaptation. It extends them with a versioned contract for the exact knowledge, authority, evaluation, evidence, and projection commitments used in a capability execution and connects that contract to heterogeneous CEEs and governed evidence disposition.

2.3 Runtime models, model management, and evolution

Models@run.time keeps an explicit model available during operation so that a system can reason about its current state and adapt (Blair et al. 2009). Work on business-process models at runtime extends this principle toward operational process representations (Redlich et al. 2014). ICLA shares the concern that models remain operationally relevant after design time, but differs in two respects. First, the Registry and CKCs describe institutional authority and commitments rather than a causally connected representation of the internal state of a running system. Second, a CEE can execute autonomously between the delivery of an admitted assembly and the submission of selected evidence; ICLA is not a control loop over every internal execution step.

Megamodels and generic model management make heterogeneous models and their relations explicit and support operations over model collections (Salay et al. 2020). Ontology-evolution research identifies change needs from system use, while historized runtime models make versions and stable element identities available for history-aware reasoning (Hartmann et al. 2014; Stojanovic et al. 2003). ICLA reuses typed relations, explicit versions, and preserved history. It adds capability-specific activation by treating evolution as an authority-bearing transition: a complete successor CKC becomes authoritative only for later resolutions, while earlier assemblies retain exact references to the contracts that governed them.

2.4 Process discovery and institutional discovery

Process discovery derives behavioral process models from recorded execution events and evaluates trade-offs among fitness, precision, generalization, complexity, and scalability (Augusto et al. 2019; van der Aalst et al. 2004). It can reveal recurrent activities, ordering, and variants that were not explicit in a prior model. ICLA uses such detected patterns as proposal signals for a different transition. Crystallization subjects the candidate responsibility, ownership, value, evidence coverage, and overlap to review before governance assigns a capability identity and initial CKC. Process or pattern discovery can therefore provide signals; ICLA specifies the authority boundary by which a signal may extend the institutional capability model.

2.5 Organizational memory and distributed AI execution

Organizational-memory research explains how knowledge is retained and retrieved across people, routines, artifacts, and repositories (Alavi and Leidner 2001; Stein 1995; Walsh and Ungson 1991). Complementary knowledge-management research examines organizational knowledge creation, integration, and sharing governance (Foss et al. 2010; Grant 1996; Nonaka and Takeuchi 1995). Provenance and catalogs make sources and derivations navigable (Lebo et al. 2013; Subramaniam et al. 2021). These works motivate distributed source authority and traceable change, but do not make an enterprise capability and its execution-specific knowledge-and-evaluation commitment the principal unit of navigation.

Prior work on knowledge-centric information systems proposes a broader knowledge-architecture framing in which data-engineering guarantees are reinterpreted for heterogeneous knowledge artifacts, including semantic change detection, provenance, and consumer-ready views (Garralda-Barrio 2026). ICLA narrows that framing to capability-bound authority: source changes become impact signals, whereas only governed decisions may alter the CKC selected for future executions.

Multi-agent and human–agent systems distribute reasoning across participants with different roles, knowledge, tools, and coordination structures (Sun et al. 2025; Wu et al. 2024; Yang et al. 2025; Zou et al. 2026). Retrieval-augmented generation (RAG), context engineering, memory systems, and agent skills provide runtime selection and reuse mechanisms (Gao et al. 2026; Kang et al. 2025; Lewis et al. 2020; Li et al. 2025; Mei et al. 2025; Q. Zhang et al. 2026; Z. Zhang

et al. 2025). ICLA treats these mechanisms as possible realization choices inside assembly or execution. Recent work on embodied-agent capability evolution separates persistent agent identity from evolving, versioned executable capability modules; related lifecycle-governance work treats new module versions as governed deployment candidates subject to compatibility checking, sandbox and shadow deployment, gated activation, monitoring, and rollback (Qin et al. 2026a,b). These studies make persistent agent identity and the executable-capability lifecycle primary. ICLA instead makes the institutional capability lineage primary: a recurring institutional responsibility is the stable address, its knowledge-and-evaluation commitments are versioned across heterogeneous CEEs, and institutional evolution distinguishes CKC succession from governed formation of a new capability identity.

2.6 Comparative gap and contribution

The reviewed traditions supply most individual concepts required by ICLA: capability, goal, context, KPI, runtime representation, model relation, provenance, version, discovery, and change. The gap is relational rather than elemental: they do not make the authority-preserving connection the central conceptual object. In ICLA, a stable capability identity selects an immutable CKC; that contract governs an intent-specific projection for a heterogeneous CEE; and only qualified evidence and an authorized decision may change the contract selected for future executions or establish a new capability identity.

On this basis, ICLA defines *incremental evolution* as governed extension of retained institutional state in two dimensions: deeper contract history under an existing identity and controlled addition of a newly authorized capability identity. The comparison also distinguishes enterprise capability, running system or process, model collection or schema, repository, execution instance, and institutional lineage. Tab. 1 states the inherited strengths and boundary of the claimed contribution.

This comparison provides the basis for examining and justifying the architectural requirements in Sect. 4: distributed knowledge retains source authority; institutional responsibility remains addressable through versioned contracts; CEEs preserve situated autonomy and an explicit execution boundary; projections remain contextual and reproducible; and returned evidence enters a controlled evolution path.

3 Research Method

This study reports design-science research (DSR) refinement and evaluation of an existing problem-driven artifact design (Hevner et al. 2004; Peffers et al. 2007). ICLA's kernel comprises a capability-lineage meta-model, typed relations, protected boundaries, explicit transitions, and proposed conformance invariants. Logical interfaces, schemas, and reference records represent it; a deterministic implementation exercises resolution, evidence handling, successor activation, and governed capability formation with separate initial activation. We use the activity vocabulary of Peffers et al. (2007) to report problem identification, objectives, design, demonstration, evaluation, and communication as iterative activities. Tab. 2 identifies their methods, evidence, and outputs.

3.1 Responsibility-oriented literature synthesis

The identified problem and initial design were examined through a purposive, responsibility-oriented integrative synthesis across conceptual and enterprise modeling, capability modeling, runtime models, model evolution, process discovery, provenance, organizational memory, and DSR. Seed works were expanded through targeted searches, limited backward citation chaining, and bibliographic verification. The synthesis supports architectural justification, boundary refinement, and positioning rather than exhaustive coverage.

Sources were retained when they supplied an artifact construct, represented an adjacent design, or established an authority, evidence, or evolution constraint. Analysis considered the modeled object, protected boundary, governing authority, and transition or failure requiring explanation. Novelty

Table 1: Prior modeling traditions and the delimited contribution of ICLA.

Tradition	*Strength used by ICLA*	*Primary concern in prior work*	*ICLA's delimited extension*
Multi-perspective enterprise modeling (Fill 2018; Frank 2014)	Integrated perspectives, meta-modeling, and extensible semantics	Organizational and information-system analysis through modeling languages and annotations	A capability-lineage meta-model with execution, evidence, and activation records
Capability-driven development (Koutsopoulos et al. 2024, 2021; Stirna et al. 2012; Zdravkovic et al. 2013)	Capability, goal, KPI, context, realization, relation, and adjustment	Context-dependent capability design, delivery, change, and stakeholder use	Versioned knowledge-and-evaluation commitments and evidence-governed succession across heterogeneous CEEs
Models@run.time and runtime process modeling (Blair et al. 2009; Redlich et al. 2014)	Operationally available models and adaptation reasoning	Current system or process state and its adaptation	Institutional authority snapshots without governing each internal execution step
Megamodel and model evolution (Hartmann et al. 2014; Salay et al. 2020; Stojanovic et al. 2003)	Typed relations, heterogeneous artifacts, versions, identifiers, and controlled change	Management and evolution of models, schemata, ontologies, or runtime histories	Capability-specific active pointers, immutable execution references, and decision-linked successor deltas
Process discovery (Augusto et al. 2019; van der Aalst et al. 2004)	Detection of recurrent behavior and derivation of process models from execution records	Behavioral structure, variants, and conformance in observed processes	Governed formation of a recurrent responsibility supported by retained evidence of realized work or unmet demand, with identity, owner, lifecycle, and initial CKC
Organizational memory and provenance	Distributed retention, source authority, and derivation history	Knowledge persistence and traceability across repositories and practices	Capability as the navigation and accountability address for selected knowledge and evidence
AI context, memory, and skills	Runtime retrieval, assembly, reuse, and distributed execution	Task-time context or executor behavior	Consumer-independent CKC, explicit CEE boundary, and governed evidence disposition

was assessed through responsibility and relation coverage rather than keyword absence.

This analysis informed the objectives and protected boundaries in Sect. 4. The requirements-to-object mapping links the identified problem and prior work to the proposed artifact.

3.2 DSR activities and outputs

Tab. 2 links each DSR activity to the method applied, the evidence used, and the resulting artifact output, thereby separating performed research activities from prospective organizational validation.

This mapping follows prior work on representing DSR processes (Gau et al. 2023). It documents the study's iterations without prescribing a sequence or evaluation method for other projects.

3.3 Iterative design and boundary decisions

Artifact development proceeded through connected modeling refinements rather than one pass from requirements to implementation. Construction of the end-to-end trace exposed dependencies among requirements, equations, schemas, and invariants. Tab. 3 combines the resulting design

Table 2: Reported DSR activities, methods, evidence, and outputs.

DSR activity	*Method applied*	*Evidence or material used*	*Output in this paper*
Problem identification and motivation	Problem articulation and responsibility-oriented comparative positioning	Identified continuity problem, initial artifact design, and conceptual, capability, runtime, evolution, provenance, discovery, memory, and AI execution literature	Research question, refined problem framing, and comparative gap in Sects. 1–2
Objectives	Examination and refinement by protected boundary and accountable transition	Initial artifact design, identified conditions, authority conflicts, versioning needs, and heterogeneous execution conditions	Justified objectives and requirement-to-component mapping in Tab. 4
Design and development	Conceptual modeling, typed relation design, explicit specification, and architecture allocation	Capability, CKC, assembly, execution, evidence, decision, and lifecycle objects	Meta-model, equations, planes, interfaces, invariants, and conformance profiles
Demonstration	Constructed reference scenarios	OAuth execution-to-succession fixture; submitted-proposal formation fixture with retained-history references, new identity, CKC v1, and separate activation	Linked traces in Sect. 10 and companion
Evaluation	Analytical structural assessment, schema validation, trace replay, negative tests, and invariant checking	Requirements, equations, nine schemas, linked records, deterministic implementation, and executable tests	Assessment in Sect. 11 and anonymized companion artifact
Communication	Architecture description and reproducible reference kit	Manuscript, logical interfaces, implementation mapping, and versioned companion specification	This paper and the submitted companion artifact

problems, rejected alternatives, retained refinements, and architectural effects.

Together, these refinements establish the identity, authority, projection, autonomy, and evolution boundaries carried into the architectural requirements and formal model.

3.4 Demonstration and evaluation strategy

The OAuth 2.1 and capability-formation materials are constructed reference scenarios for artifact assessment. OAuth exercises normal and negative execution-to-succession paths; formation begins from a submitted proposal supported by retained history references and exercises identity assignment, initial CKC v1 append, and separate activation. The demonstration tests representation of both governed transitions and invariant preservation by the reference implementation within its stated conformance scope. Prospective studies address discovery accuracy, institutional judgment quality, practitioner acceptance, organizational benefit, and comparative performance.

The evaluation strategy combines formative and summative artificial evaluation, consistent with DSR evaluation guidance (Prat et al. 2015; Venable et al. 2016). Formative checks include requirement-to-component tracing, equation–schema alignment, identifier consistency, and negative cases such as inadmissible resolution and non-standard evidence. Summative artificial evaluation uses the constructed traces, machine-checkable schemas and linked records, deterministic replay and conformance

Table 3: Trace of major design refinements and retained boundary decisions.

Problem or rejected alternative	*Retained refinement*	*Architectural effect and rationale*
Copy governed knowledge into a central capability store	Retain source authority in distributed repositories and record versioned bindings	Avoids a second authoritative content copy while preserving impact paths
Couple capability identity to a mutable current contract	Separate stable Registry identity from immutable CKC versions selected through an active pointer	Preserves a stable address and interpretable contract succession
Mix canonical commitments with executor-specific context	Introduce an immutable contextual assembly and substrate-specific materialization	Separates contract from projection and preserves reproducible execution inputs
Require central interaction for every reasoning or coordination step	Admit a bounded assembly and permit CEE-local execution	Preserves operational autonomy; ICLA interaction resumes for re-resolution or evidence submission
Allow accepted executor output to update canonical state directly	Introduce evidence qualification, adjudication, and authorized activation	Separates knowledge production from institutional authority
Use one evolution path for successor contracts and recurrent responsibilities	Separate CKC succession from capability crystallization and require proposal review before identity assignment	Preserves distinct identity and ownership consequences and prevents automatic promotion

tests, and structural criteria including fidelity, declared-scope coverage, consistency, traceability, and technical instantiation. Sect. 11 reports these results separately from the prospective comparative protocol required to test organizational effectiveness and governance cost.

Throughout the paper, *demonstration* denotes scenario instantiation, *formative evaluation* denotes refinement checks, and *summative artificial evaluation* denotes post-design assessment. *Prospective comparative evaluation* denotes the unexecuted protocol. *Organizational validation* is reserved for future independent evidence of utility or effectiveness, while technical uses such as schema validation retain their qualifier.

4 Design Objectives and Architectural Requirements

We make the artifact's responsibilities and protected boundaries explicit and justify them against the identified problem and literature gap, following architecture-description principles that represent concerns, viewpoints, and relations (ISO/IEC/IEEE 2022). Fig. 1 presents the resulting architecture at its simplest; Tab. 4 traces each component to the boundary it protects.

Five objectives guide artifact design and architectural evaluation; organizational improvement remains part of the prospective evaluation agenda.

O1—preserve institutional continuity. A recurring responsibility needs an address that remains interpretable when performers, software, repositories, and delivery configurations change. ICLA therefore separates stable capability identity, immutable CKC versions, and the Registry pointer that selects current authority.

O2—make capability commitments explicit. Capability metadata identifies and describes a recurring responsibility, but it does not define the

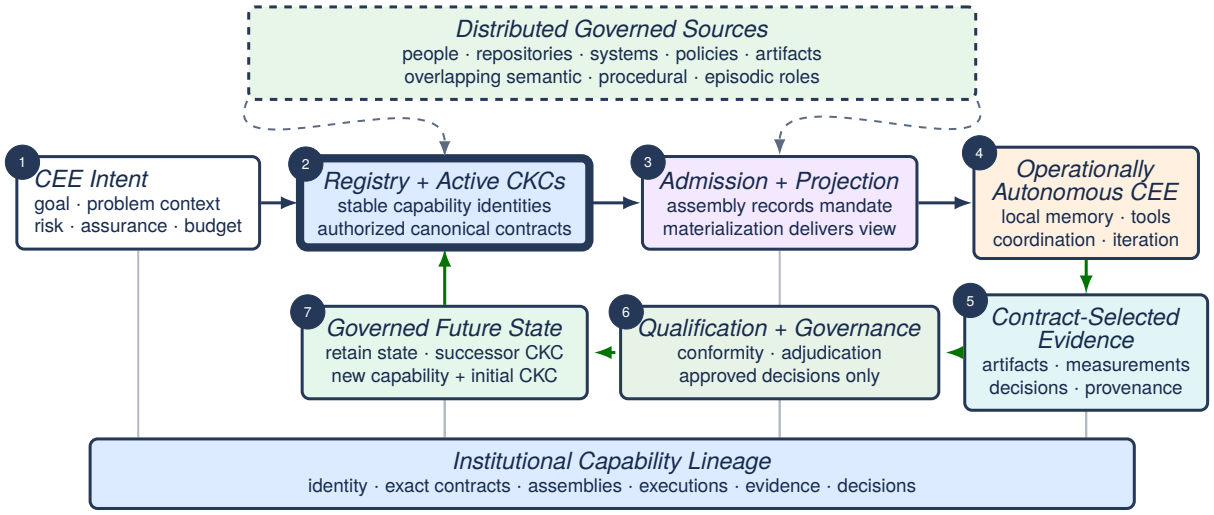


Figure 1: ICLA block architecture. A CEE intent resolves to one or more active capability contracts. Admission establishes a bounded mandate, the assembly records its limits, and materialization delivers the execution view to an operationally autonomous CEE. Contract-selected evidence returns to governance for retention, CKC succession, or capability crystallization; the lineage preserves the governed trace.

complete authorized basis for performing it: applicable source bindings, obligations, authorities, evidence and evaluation rules, and projection rules. A CKC records and jointly versions these commitments; the governed source content itself remains in its authoritative repositories.

O3—support situated heterogeneous execution. A capability may be performed by a person, team, workflow, service, agent configuration, or hybrid. Consumer-independent commitments must therefore be projected into an intent-specific assembly and delivered without prescribing one executor representation or internal memory.

O4—separate knowledge production from institutional change. Sources and executions produce candidate observations, artifacts, measurements, and lessons. The architecture must retain their provenance, qualify them against the evidence contract, and require an accountable decision before they alter current institutional state.

O5—support incremental evolution without rewriting history. Earlier assemblies and decisions must remain explainable against the source and contract versions that governed them. CKC succession must refine an existing capability through explicit successor records and deltas, while crystallization must add a separately governed identity and initial CKC. Neither transition may rewrite retained lineage.

The objectives are expressed as requirements through a condition–responsibility–object chain. A condition identifies what can vary or conflict, a responsibility states what must be preserved or controlled, and an ICLA object is introduced when it protects a distinct boundary. Internal coverage, linked records, and negative cases assess this traceable design argument; prospective comparative evaluation addresses organizational effectiveness.

5 Core Conceptual Model

This section defines the static vocabulary of ICLA before Sects. 6 and 7 explain execution and change. It first separates institutional, execution, and transition state; then defines capability identity, the Registry, CKCs, governed sources, and the records that connect situated use to institutional authority. The closing authority rules state which objects persist, which may change, and who may authorize each transition.

Table 4: Traceability from architectural requirements to ICLA components.

Identified condition	*Required responsibility*	*ICLA object and protected boundary*
Distributed sources, policies, authorities, and interpretations change independently	Expose exact versions and bindings; detect content, semantic, authority, policy, and temporal-validity changes; propagate impact without moving source authority	Organizational Memory and impact records; distributed source governance and governed-change boundary
Responsibilities outlive CEE configurations and implementations	Preserve stable identity, ownership, relations, lifecycle, and active authority	Registry; institutional continuity
Capability metadata cannot express the required commitment	Define the canonical contract for knowledge, evaluation, evidence, authority, and projection	CKC; explicit contract semantics
Heterogeneous execution requires situated views and delivery forms	Establish a bounded operational mandate while preserving local autonomy and traceability	CEE, Assembly, and Materialization; contract/projection boundary
Sources and executions produce candidate institutional inputs	Qualify, adjudicate, and activate change without direct canonical mutation	Evidence Gateway and Governance; governed evolution
Repeated work or unmet demand may reveal an unmodeled responsibility	Review boundary, ownership, value, and overlap before assigning identity	Crystallization; governed capability formation

5.1 Model scope and logical planes

The model separates canonical institutional state, execution-specific state, and governed transition records. Canonical state comprises stable capability identities, immutable CKC versions, and governed source bindings; authoritative content remains distributed. Execution state comprises resolutions, assemblies, materializations, and evidence. Authorized transitions may change current authority without altering retained objects.

Fig. 2 presents these object types and their cardinality-bearing relations. The equations specify their signatures, constraints, and authorized transitions for conformance checking; a complete behavioral semantics lies outside the artifact.

ICLA separates three logical planes. The *knowledge plane* holds distributed sources, evidence, CKC representations, and change signals; the *control plane* holds Registry state, resolution, policy, lineage, governance, and discovery; and the *distributed execution plane* contains CEEs that receive materializations, act with local resources, and return evidence. Coordination may be centralized, peer-to-peer, or hybrid; Registry interaction resumes for re-resolution or evidence submission.

5.2 Institutional Capability and Registry

Eq. (1) models the Registry as a time-indexed attributed graph:

$$\Gamma_t = \langle C_t, \mathcal{E}_t, \mu_t, \nu_t \rangle, \tag{1}$$

Here, C_t is the set of institutional capability identities and $\mathcal{E}_t$ is the set of typed relations. The function μ_t maps capabilities to metadata. Let $C_t^{\text{act}} \subseteq C_t$ denote capabilities whose lifecycle state permits new resolution; ν_t is defined on that subset and maps each such capability to its active CKC version. Retired or pre-activation identities remain in C_t and their lineages without requiring a current pointer.

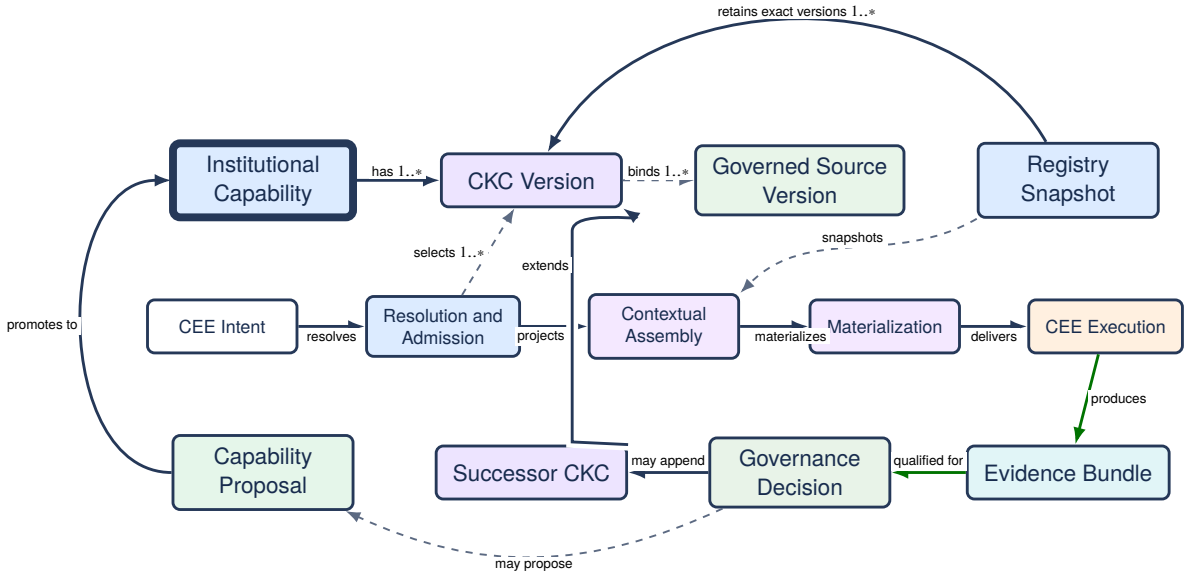


Figure 2: Core ICLA conceptual meta-model. Stable capability identity, immutable contract versions, contextual assemblies, substrate-specific materializations, execution records, evidence, and governed transitions are distinct object types connected by typed and cardinality-bearing relations. The successor delta is retained within the Governance Decision record.

Relations such as *depends on*, *specializes*, *replaces*, and *shares knowledge with* support resolution, impact analysis, and lifecycle management. For each $c \in C_t$, $\mu_t(c)$ includes identity, outcome, owner, domain, lifecycle, risk, maturity, and policy. Authorized actors govern this metadata and the CKC pointers; resolvers filter it by intent, CEE, conditions, risk, and policy before following source bindings. The Registry therefore indexes distributed knowledge rather than duplicating it.

Tab. 5 defines the relation types used in resolution, assembly, impact analysis, and lifecycle management.

5.3 Capability Knowledge Contract and Lineage

Eq. (2) defines CKC version v for capability c:

$$K_c^v = \langle S_c^v, R_c^v, E_c^v, Q_c^v, G_c^v, A_c^v \rangle, \qquad (2)$$

Here, S defines scope and source bindings; R requirements, obligations, and relations; E evidence schemas; Q evaluation; G governance and authority; and A projection rules. Together, these commitments form one contract because their jointly versioned snapshot defines the complete authorized basis for resolving, projecting, executing, evaluating, and governing changes to a capability's commitments. They may be implemented through separate but linked schemas, policies, or services; the CKC preserves their common identity and version boundary without prescribing an executable policy format or storing authoritative content.

Completeness is assessed through representative cases, obligations, and validators in Q and G. Canonicality is an authority property: K_c^v is a complete immutable contract version authorized within capability c's governed lineage. It becomes the contract selected for new resolutions only when the active pointer references it; its sources remain distributed and assemblies remain derived.

Eq. (3) distinguishes the living contract lineage from its immutable canonical versions:

Table 5: Institutional capability relation types.

Relation	*Meaning*	*Operational use*
`depends_on`	Capability requires another capability	Impact analysis and mandatory assembly inclusion
`composes_with`	Capabilities are frequently used together	Candidate assembly templates and crystallization signals
`specializes`	Capability narrows another capability	Inheritance and override rules
`replaces`	Capability supersedes another	Migration, deprecation, and compatibility
`shares_knowledge`	CKCs reference overlapping sources	Change propagation and duplication control
`derived_from`	Capability originated from another capability or supporting retained history	Crystallization lineage
`conflicts_with`	Simultaneous application needs resolution	Assembly validation and governance review

$$\mathcal{L}_c(t) = \langle K_c^1 \prec K_c^2 \prec \cdots \prec K_c^{n_c(t)} \rangle,$$
$$\nu_t(c) = K_c^{a_c(t)}, \qquad c \in C_t^{\text{act}},$$
$$1 \le a_c(t) \le n_c(t). \tag{3}$$

Here $\prec$ denotes approved succession, $n_c(t)$ is the latest governed version of capability c, and $a_c(t)$ is its active version index. Thus, $a_c(t) < n_c(t)$ represents a governed successor that is not yet active. Sect. 7 defines successor append, its decision-linked delta, and the separate activation effect; earlier versions and assemblies remain immutable.

The CKC lineage is the contract-succession substructure within the broader *institutional capability lineage*. Eq. (4) models the broader lineage as a Registry-addressable typed relational structure:

$$\Lambda_c(t) = \langle \mathcal{V}_c(t), \mathcal{R}_c(t) \rangle, \tag{4}$$

where $\mathcal{V}_c(t)$ contains capability c, exact CKCs, assemblies, executions, evidence, decisions, and lifecycle records, and $\mathcal{R}_c(t)$ their typed links. A *lineage trace* is a concrete retained subgraph of this structure for specified executions and transitions. A retained record belongs to the governed lineage when a typed ICLA relation connects it to c under the applicable retention policy.

5.4 Governed Sources and Organizational Memory

Let $\mathcal{M}_t$ denote versioned documents, code, policies, decisions, data, interpretations, and evidence across governed repositories at time t. Their semantic, procedural, and episodic roles may overlap. Organizational Memory exposes versions, bindings, retention, temporal validity, and change events while source owners retain content authority. A relevant change may concern content, meaning, authority, policy, dependency, or validity. It initiates traceable impact analysis over affected bindings, capabilities, CKCs, assemblies, and CEEs, but does not directly mutate an active CKC or rewrite an earlier assembly.

5.5 Execution, Governance, and Authority Records

The remaining classes connect situated use to possible institutional effects; decisions and proposals record possible change without granting execution artifacts institutional authority. Sects. 6 and 7 define their behavior.

Tab. 6 closes the static view by distinguishing retention, mutability, temporal state, and change authority for each information object.

Table 6: Persistence, mutability, temporality, and authority rules.

Object	*Retention basis*	*Mutation / temporal state*	*Authority and permitted transition*
Capability identity, metadata, pointer	Current state and policy-protected lineage	Identity fixed; metadata versioned; pointer governed	Formation assigns identity; governance changes lifecycle, relations, or pointer without reassigning identity
CKC version and lineage	Protected while referenced or required for interpretation	Version immutable; lineage append-only; latest may be inactive	Governance appends complete successor, delta, and decision; distinct activation updates the pointer
Resolution / admission snapshot	Policy-protected resolution record	Immutable after outcome	Resolver freezes Registry state, any admitted CKC map, rationale, and status
Contextual Assembly	Protected per retained execution	Immutable after creation	Assembly service derives the admitted commitments without changing their semantics or authority
Materialization	Payload policy-dependent; lineage retained	Frozen or content-addressed	Delivery service adapts representation to the CEE substrate without changing assembly semantics or authority
CEE execution state	Boundary record retained per policy; local memory local unless submitted	Record append-only; working state CEE-controlled	CEE controls local state; submissions enter evidence without institutional authority
Evidence bundle	Protected when submitted	Append-only qualification history	Producer submits; Gateway validates conformity; governance determines disposition
Governance decision	Protected decision record	Append-only or explicitly superseded	Authorized reviewers record disposition and separately identifiable canonical effects
Capability proposal	Protected when policy requires	Mutable until submission; then versioned	Detection or actors propose; only promotion assigns identity and initial CKC
Source artifact	Source-policy dependent	Versioned	Source owner controls content; Registry records bindings and impact paths

6 Situated Capability Execution

A situated execution instantiates governed capability state for one admitted intent: resolution selects exact active CKCs, assembly projects them, materialization delivers the usable representation, and the CEE returns contract-governed evidence.

6.1 CEE Boundary and Intent

A *Capability Execution Environment* (CEE) is the situated boundary that enacts an admitted capability instance and may return selected outputs as contract-interpretable evidence. The boundary may be instantiated by a person, team, agent configuration, workflow, service, or hybrid; participation in execution alone does not constitute that boundary. In AI-enabled work it may be a reconfigurable human–machine arrangement. Participants form one composite boundary when they share a mandate, execution identifier, and evidence contract; differences in authority, assurance, retention, or evidence obligations require separate admission.

The record contains attributes needed for resolution, authorization, assurance, traceability, and evidence comparison. The boundary reference η identifies the admitted configuration and authority perimeter, while e identifies a concrete execution occurrence within that perimeter. Neither is a persistent institutional capability identity. Changes in tools, participants, agents, or topology remain local unless they affect recorded attributes and therefore require a new boundary snapshot or re-resolution.

Admission fixes the mandate; materialization delivers the view. Within that mandate, the CEE controls logic, state, tools, artifacts, and coordination. Realization may be deliberative or determin-

istic; operational autonomy grants no institutional authority. Each execution links boundary, configuration, intent, assembly, and evidence; the Gateway checks conformity and governance decides disposition and activation.

An intent is $i_\eta = \langle g, x, \eta, u, r, b, q \rangle$, where g is the goal, x the problem context, η the situated CEE boundary, u the consumer and delivery configuration, r risk, b budget, and q required quality or assurance. Its semantic content, $\langle g, x \rangle$, states the requested outcome and problem conditions used to identify an institutional responsibility; the remaining fields constrain situated admission and realization. Resolution evaluates both, while the CEE record supplies execution context.

6.2 Resolution and Admission

Eq. (5) maps the intent to registered capabilities and retains the exact contract selection:

$$\rho_t(i_\eta, \Gamma_t) \to \langle C_i, \kappa_i, \sigma_i, \omega_i, \pi_i \rangle, \qquad (5)$$

where $C_i \subseteq C_t$ is the selected set; $\kappa_i(c) = \nu_t(c) = K_c^{a_c(t)}$ its exact active CKC map; σ_i the Registry snapshot; ω_i ranking and confidence; and π_i rationale. Resolution snapshots the CEE boundary; later pointer changes do not alter κ_i. ICLA leaves semantic representation and equivalence to rule-, graph-, model-based, or hybrid matching. Audit records retain input, candidates and exclusions, selected map, Registry and matcher versions, rationale, confidence, and outcome, separating traceability from matching quality.

Candidate generation matches semantic content to Registry metadata; relation expansion follows mandatory dependencies; and constraint validation evaluates the remaining intent fields against Registry, CKC, and policy state to remove unauthorized, retired, conflicting, stale, or assurance-incompatible candidates. Without an applicable active CKC or admissible composition, resolution MUST NOT invent a contract or produce an authoritative assembly. The retained gap may inform CKC succession or, if recurrent and unregistered, a pre-institutional crystallization proposal. Admitted resolutions receive assembly records.

An admitted intent may resolve to one CKC or require several because no single registered capability covers the requested outcome. Eq. (6) represents the selected institutional capability set:

$$C_i = \{c_1, c_2, \ldots, c_k\}, \quad k \geq 1. \qquad (6)$$

Assembly combines the selected contracts and bound sources while preserving per-contract authority, provenance, evaluation obligations, and resolved conflicts. Later pointer changes do not alter the retained selection, and reused patterns still produce intent-specific records. Failed correctness or assurance yields failure or escalation.

6.3 Contextual Assembly and Materialization

For an admitted resolution, Eq. (7) derives a view from the intent and selected CKCs:

$$\Pi_{i,\eta} = \alpha\left(i_\eta, \kappa_i, \sigma_i, \mathcal{M}_t, P_t\right), \qquad (7)$$

where κ_i is the exact capability-to-CKC map and σ_i the Registry snapshot retained by the admitted resolution; P_t contains assembly and security policies. Passing the map rather than only its values preserves which capability authorized each selected contract. $\Pi_{i,\eta}$ records the situated CEE boundary, retained Registry snapshot, exact selected CKCs, selections, exclusions, provenance, and their evidence and evaluation contracts. Materialization adapts this immutable projection to substrate s as $B^s_{i,\eta} = \text{materialize}(\Pi_{i,\eta}, s)$. Bundles, workspaces, messages, procedures, or governed access handles retain the same lineage and authority bounds; materialization neither changes assembly semantics nor acquires canonical authority. Material changes to the intent's semantic content, CEE boundary, authority, freshness, risk, or assurance require re-resolution.

6.4 Assembly Correctness

For an admitted resolution with selected capability set C_i, Eq. (8) verifies the correctness of the resulting assembly as a conjunction:

$$\begin{aligned}\mathrm{Valid}(\Pi_{i,\eta}) = \ & \mathrm{Traceable}(\Pi_{i,\eta}) \\ & \wedge \mathrm{Authorized}(\Pi_{i,\eta}) \\ & \wedge \mathrm{RequiredCovered}(\Pi_{i,\eta}, C_i) \\ & \wedge \mathrm{EvaluationBound}(\Pi_{i,\eta}) \\ & \wedge \mathrm{ConflictsResolved}(\Pi_{i,\eta}) \\ & \wedge \left[\mathrm{Budget}(\Pi_{i,\eta}) \leq b\right].\end{aligned} \tag{8}$$

Assembly validity requires traceability, authorization, coverage, evaluation binding, conflict resolution, and budget compliance. RequiredCovered evaluates CKC-declared cases, obligations, validators, and mandatory relations. Its trace records the applied method—deterministic, model-assisted, or human-reviewed—and applicable validator, model, or review-policy version.

6.5 CEE Execution and Evidence Submission

Evidence from execution e is a tuple of results, artifacts, governed metrics, incidents, execution-local decisions, lessons, and provenance:

$$O_{e,\eta} = \langle R_e, A_e, M_e, I_e, D_e^{\mathrm{loc}}, L_e, P_e \rangle.$$

M_e reports governed metric identifiers, values, units, collection conditions, aggregation, and uncertainty. D_e^{loc} holds local decisions or judgments; unlike governance disposition d_t, they have no institutional authority. Evidence traces from the CEE boundary and intent to capabilities, exact CKCs, assembly, materialization, policies, sources, and any versioned transformation used to construct the submitted report. It remains a candidate contribution until adjudication; undeclared measurements remain non-standard.

Submission closes the situated execution path; Sect. 7 defines how qualified evidence may affect institutional state.

7 Governed Institutional Evolution

Governance determines whether source changes or submitted evidence remain local, extend an existing CKC lineage, or justify a new institutional capability.

7.1 Qualification and Adjudication

Source changes and submitted outputs never rewrite canonical state directly. They remain lineage records whether rejected, retained, or used to update bindings, append a successor CKC, or propose a new capability; production or recurrence alone grants no institutional authority.

Let z_t denote an institutional maintenance event: a source change affecting content, meaning, authority, validity, or version; qualified CEE evidence $O_{e,\eta}$; a policy revision; or an ownership or lifecycle event. Let $\mathcal{K}_t$ denote the applicable CKC versions and $\mathcal{G}_t$ the applicable governance rules and authorities. Eq. (9) evaluates the event's institutional consequences:

$$J_t(z_t, \Gamma_t, \mathcal{K}_t, \mathcal{G}_t) \to d_t, \tag{9}$$

where d_t records disposition after event validation and, for execution evidence, evaluation-contract measurement checks. It may combine compatible effects on affected objects: reject, quarantine, or retain; authorize binding or rule updates or successor construction and append for affected CKCs; revise Registry state; or initiate a capability proposal.

1. *Qualification*: Is the event relevant, sufficiently grounded, and complete enough for review?
2. *Conformity*: Do the change record, evidence, and measurements satisfy schemas, metric identifiers, units, collection rules, policies, provenance, and authorization?
3. *Adjudication*: Should the institution reject, retain, postpone, or promote the change?
4. *Propagation*: Which sources, CKCs, relations, caches, and future assemblies are affected?

The versioned path supports documented governance and risk management (Tabassi 2023). Detection may open review automatically, but only governance can alter active Registry state.

7.2 CKC Succession, Append, and Activation

When d_t authorizes successor construction, Eq. (10) derives the complete successor:

$$K_c^{v+1} = \text{evolve}(K_c^v, z_t, d_t). \quad (10)$$

The decision-linked successor delta is:

$$\Delta_c^{v \to v+1} = \text{diff}(K_c^v, K_c^{v+1}, z_t, d_t). \quad (11)$$

$\Delta_c^{v \to v+1}$ records the changed CKC commitments among S, R, E, Q, G, and A, together with the rationale, supporting evidence, authorizing decision, and rollback reference. It explains the transition but is not required to reconstruct the complete successor CKC. An approved succession decision authorizes construction and append of the complete successor and its delta; it does not by itself change the active pointer. Retained assemblies preserve their original snapshot.

Successor creation and pointer activation are distinct transitions. A complete successor may therefore exist in $\mathcal{L}_c$ without governing new resolutions. For an already appended version K_c^w, activation is the separately auditable pointer update

$$\text{activate}_t(c, K_c^w, d_t^{\text{act}}) \to \nu_{t+1}(c) = K_c^w, \quad (12)$$

where d_t^{act} is an authorized activation decision. Creation and activation may be approved in one governance transaction, but their records and effects remain distinguishable.

7.3 Capability Crystallization and Formation Lifecycle

For retained history H_t, Eq. (13) returns pre-institutional proposal records in candidate state:

$$\chi(H_t) \to \mathcal{D}_t. \quad (13)$$

Each $p_j \in \mathcal{D}_t$ begins in candidate state and may be enriched and submitted for review with the recurrent combination, stable assembly rules, value, candidate owner, overlap analysis, and draft CKC. Candidate and submitted proposal are lifecycle states of the same pre-institutional record, not distinct object types. The record has neither an institutional capability identity nor institutional status before review. Governed promotion is defined only for a submitted proposal $p_j \in \mathcal{D}_t$ and an authorized decision d_t recording the configured ownership, distinctiveness, overlap, value, and evidence review. Its result follows Eq. (14):

$$\text{promote}(p_j, d_t) \to \langle c', K_{c'}^1, \Gamma_{t+1} \rangle. \quad (14)$$

Eq. (14) identifies the objects produced by an authorized promotion; Eq. (17) later expresses the effect of that same transition on institutional state and retained lineage, not a second governance decision. Promotion assigns the identity and appends the initial immutable CKC as one identity-forming transition; activation for future resolutions remains separately governed. Rejected, deferred, or template-retained proposals remain pre-institutional and do not change C_t.

Crystallization uses supporting retained history—realized work, non-admitted resolution outcomes, or other authorized discovery records—to identify stable responsibilities absent from the capability model. Such signals establish neither identity nor promotability; recurrence concerns a stable responsibility across intents, not repeated wording or executor configuration. Formation additionally requires:

- repeated demand across a declared time horizon;
- a stable responsibility distinct from existing institutional capabilities;
- a coherent and reusable assembly signature;
- measurable value or risk reduction under declared evaluation contracts;
- candidate ownership and governance authority;
- sufficient source and evidence coverage;
- manageable overlap and conflict with existing capabilities.

These are configurable review predicates rather than universal numerical thresholds.

Fig. 3 locates these pre-institutional states relative to the governed formation transition and the subsequent capability lifecycle.

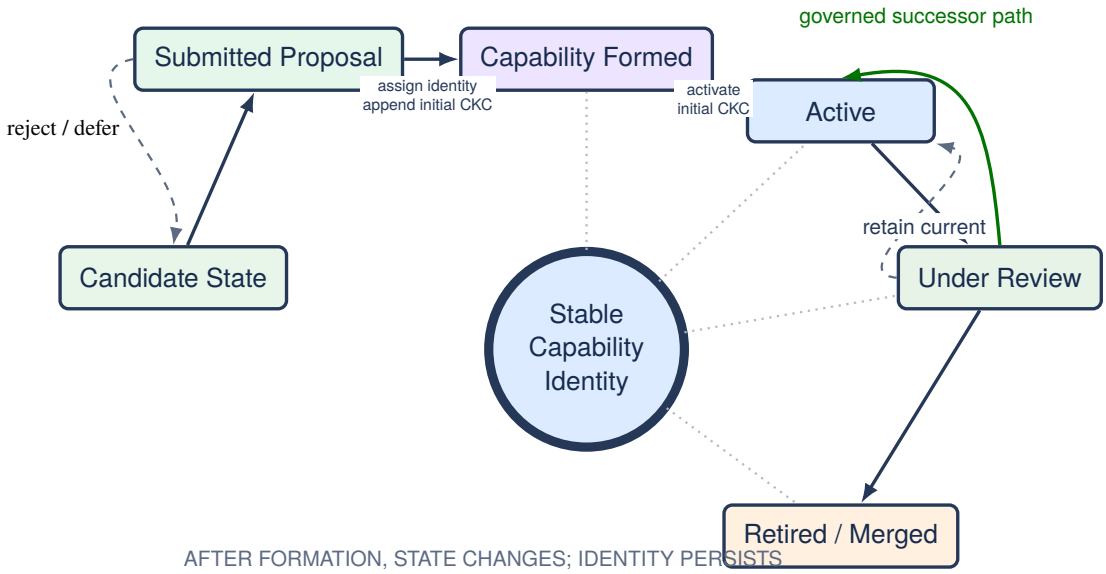


Figure 3: Capability formation and lifecycle. Candidate and submitted-proposal states belong to one pre-institutional proposal record. Governed promotion assigns the Registry identity and appends the initial CKC; activation remains a separate effect. Thereafter, capability state and active contracts may change while identity persists through review, succession, and retirement.

Proposals and decisions retain review criteria and outcomes; ICLA fixes effects, not a universal decision rule. Capabilities may also arise from strategy, process analysis, or onboarding; later transitions remain governed, retirement preserves lineage, and policy may permit reversal.

7.4 Two-Dimensional Evolution and Governance Paths

ICLA supports incremental institutional evolution through two non-equivalent transitions. CKC succession refines the governed commitments of an existing capability while preserving its identity. Capability crystallization extends the institutional capability model by assigning a new identity and initial CKC to a recurrent responsibility that has passed governed review. Figs. 4 and 5 show, respectively, the structural effects and governance paths of the two transitions.

Let the institutional model state at time t be:

$$\mathcal{I}_t = \langle \Gamma_t, \{\mathcal{L}_c(t) \mid c \in C_t\}, \{\Lambda_c(t) \mid c \in C_t\} \rangle . \tag{15}$$

An approved CKC succession produces:

$$\begin{aligned} \mathcal{I}_t &\xrightarrow{\text{succession}(c,d_t)} \mathcal{I}_{t+1}, \\ C_{t+1} &= C_t, \\ \mathcal{L}_c(t+1) &= \mathcal{L}_c(t) \parallel \langle K_c^{n_c(t)+1} \rangle, \\ \nu_{t+1}(c) &= \nu_t(c), \\ \Lambda_c(t+1) &= \text{extend}\big(\Lambda_c(t), K_c^{n_c(t)+1}, \\ &\qquad \Delta_c^{n_c(t)\to n_c(t)+1}, d_t\big). \end{aligned} \tag{16}$$

This append preserves the capability set, identity, active pointer, and retained subgraph while extend adds the successor, delta, decision, and typed relations. Eq. (12) may update the pointer immediately afterward or at a later authorized time.

Approved crystallization produces:

$$\begin{aligned} \mathcal{I}_t &\xrightarrow{\text{crystallize}(p_j,d_t)} \mathcal{I}_{t+1}, \\ C_{t+1} &= C_t \cup \{c'\}, \\ \mathcal{L}_{c'}(t+1) &= \langle K_{c'}^1 \rangle, \\ \Lambda_{c'}(t+1) &= \text{init}(c', K_{c'}^1, p_j, d_t). \end{aligned} \tag{17}$$

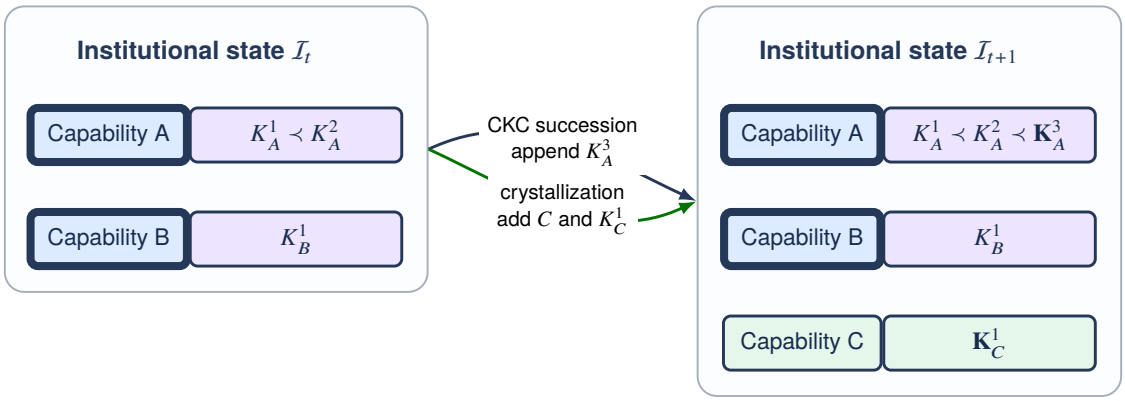


Figure 4: Two independently authorized increments shown together. CKC succession deepens an existing contract lineage without changing capability identity; crystallization adds a new identity and initial CKC. Both preserve the retained history.

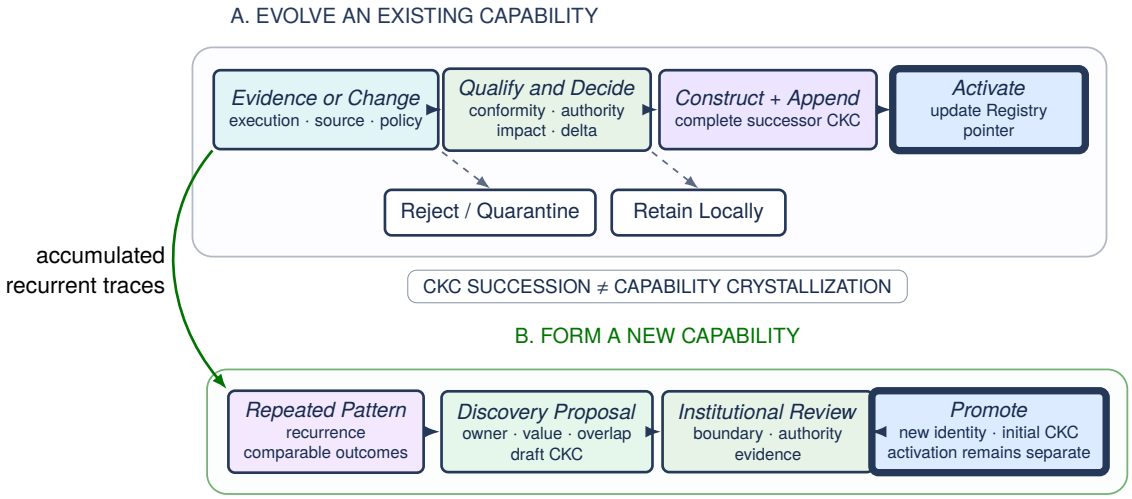


Figure 5: Decision paths for two-dimensional evolution. CKC succession appends a complete immutable version before any activation effect; its delta remains with the decision. The illustrated crystallization path uses recurrent execution traces, while other retained history may support its proposal. Review precedes formation of the identity and initial CKC; activation remains separate.

Here init creates the initial typed lineage linking the new identity and CKC through `derived_from` relations to the supporting retained history, proposal, and governance decision. Both transitions define incremental institutional evolution, but they are operationally asymmetric: succession changes commitments within an established identity, whereas crystallization crosses the identity-formation boundary. Let $\mathrm{Hist}_{\mathrm{ret}}(\mathcal{I}_t)$ denote the lineage records protected from removal because the applicable interpretation and audit policy requires them to explain retained executions or formation decisions. For authorized transitions while that retention obligation remains in force:

$$\mathrm{Hist}_{\mathrm{ret}}(\mathcal{I}_t) \subseteq \mathrm{Hist}_{\mathrm{ret}}(\mathcal{I}_{t+1}). \tag{18}$$

Incrementality is historical: state changes do not rewrite records required to explain earlier states. Append-and-pointer evolution extends affected state without reconstructing other histories or sources; resolution reads current Registry

state and active pointers without requiring lineage replay, while history retains exact references. Eq. (18) permits disposal after obligations expire if surviving references and retained executions remain interpretable.

8 Minimal Conformance Model

The preceding sections define architectural intent; the following proposed invariants make ICLA behavior observable. Within this reference architecture, *MUST*, *SHOULD*, and *MAY* use the vocabulary of Request for Comments (RFC) 2119 (Bradner 1997). They specify proposed ICLA conformance and are not externally standardized requirements.

8.1 Mandatory invariants

ICLA-1 Institutional capability identity. Every active institutional capability MUST have a stable identifier, accountable owner, lifecycle status, and governed pointer to its active CKC version.

ICLA-2 Canonical capability contract. Every active institutional capability MUST reference an immutable CKC version within a governed lineage. The version MUST declare knowledge scope, operational relations, obligations, authorities, evidence schemas, source bindings, and projection rules. Its evaluation contract MUST define applicable metrics, units, collection conditions, thresholds, interpretation rules, and the representative-case or coverage basis used for completeness claims.

ICLA-3 CEE boundary and governed authority. Every admitted execution MUST record its situated CEE boundary and the configuration attributes required for resolution, authorization, assurance, traceability, and evidence interpretation. The CEE controls internal execution logic or reasoning, working state, tools, and coordination; conformance MUST NOT require disclosure beyond the evidence contract. Its candidate contributions MUST enter through identified source or evidence paths and acquire institutional authority only through governed adjudication; no CEE MAY promote local state directly.

ICLA-4 Registry navigation. Institutional capabilities and active CKC pointers MUST be discoverable and filterable through metadata, lifecycle, policy, and conditions, and SHOULD be navigable through typed relations and source-impact paths.

ICLA-5 Intent and execution traceability. Every execution assembly MUST record the intent, its situated CEE boundary, the retained Registry snapshot, the selected capability set, and the exact CKC version mapped to each selected capability. A later active-pointer change MUST NOT alter that selection. A resolution that lacks mandatory coverage, authority, temporal validity, freshness, or assurance MUST NOT produce an authoritative assembly.

ICLA-6 Assembly lineage. Every assembly MUST trace to all exact selected CKC versions, their evaluation contracts and bound source versions, and the applicable policy and transformation versions.

ICLA-7 Contract/projection separation. Every CEE-specific materialization MUST remain traceable as a representation of its immutable assembly and MUST preserve that assembly's semantics and authority bounds. It MUST NOT be recorded or treated as a canonical CKC; reuse that changes canonical commitments MUST enter governance and produce a complete authorized CKC version.

ICLA-8 Evidence and measurement separation. CEE outputs intended to influence institutional state MUST enter through an identified submission path and MUST NOT directly mutate the active CKC or other canonical state. Reported metrics MUST reference governed definitions or be marked non-standard and excluded from direct comparison and threshold-based decisions.

ICLA-9 Governed revision and activation. Changes to source content or meaning, policy, authority, temporal validity, evidence, ownership, relations, or lifecycle that affect canonical state MUST create an impact record and governed decision. Contract changes MUST create a complete immutable successor CKC version and a decision-linked delta that records the authorized changes from its predecessor. Appending that successor

MUST NOT by itself update the active pointer. Only a separately identifiable approved activation decision or activation effect MAY update the pointer used by future resolutions.

ICLA-10 Reproducibility. A retained assembly record MUST contain enough metadata to explain or reproduce the delivered view and interpret its measurement report, subject to retention and access policies.

ICLA-11 Institutional-discovery authority. Detection of a recurrent responsibility MAY create a crystallization proposal in candidate state, but a proposal in either candidate or submitted state MUST NOT carry an assigned institutional capability identity. Promotion MUST reference a submitted proposal and an authorized review decision. Only governed, traceable promotion MAY assign a new identity in the Registry. Consistent with Eqs. (13)–(14), promotion MUST create an initial immutable CKC and preserve links to the supporting retained history, proposal, and decision from which the capability was derived. Initial activation MUST remain separately identifiable, even when promotion and activation are authorized within one governance transaction.

8.2 Conformance profiles

Tab. 7 groups the invariants into three cumulative profiles.

Conformance is claimed for a named cumulative profile and a stated assessment scope. Architectural conformance concerns the required objects, relations, transitions, and authority constraints; trace conformance concerns a concrete lineage record; and implementation conformance concerns observable behavior. Schema and deterministic checks can verify required fields, exact-version maps, referential linkage, immutability, append-only behavior, metric identifiers, pointer effects, and retained-history constraints. Institutional judgment remains necessary for the semantic adequacy of ownership and authority (ICLA-1–ICLA-3 and ICLA-9), capability coverage and conflict resolution (ICLA-2, ICLA-5, and ICLA-6), reproducibility sufficiency under policy (ICLA-10), and capability boundaries, value, and promotion (ICLA-11). A conformance claim MUST identify each invariant's assessment method and distinguish machine-checked results from governed judgments. This reporting condition governs the declaration of conformance; it is not an additional architectural invariant.

The Core profile is deliberately limited to identity, contract, projection, traceability, and reproducibility. Evidence-based canonical change begins with ICLA-Governed, while ICLA-Evolving defines the governed conformance boundary for capability discovery. These profiles stage operational adoption rather than conceptual importance: crystallization is integral to the complete ICLA model, while implementations may defer it until sufficient governed history exists.

For implementation assessment, ICLA-Evolving conformance concerns the observable proposal, promotion, lineage, and activation constraints of ICLA-11. Pattern-detection performance and the institutional quality of promotion judgments require separate empirical assessment.

9 Implementation Blueprint

Conformance constrains behavior across infrastructures. A practical deployment combines metadata and versioned object storage, existing organizational repositories, resolution and projection services, heterogeneous CEEs, an append-only Evidence Gateway, and governed review workflows. Appendix A specifies the logical interfaces and one practical implementation mapping.

9.1 A minimal implementation sequence

A pilot can defer crystallization until sufficient retained history accumulates:

1. Onboard an authorized initial Registry with stable identifiers, owners, lifecycle state, and three to five active CKCs with evaluation contracts.
2. Retain governed source knowledge in existing repositories; record bindings and ingest versioned source, policy, ownership, and lifecycle changes.

Table 7: Proposed ICLA conformance profiles.

Profile	*Required conformance scope*	*Suitable use*
ICLA-Core	ICLA-1 to ICLA-7 and ICLA-10	Governed Registry, distributed source bindings, and repeatable delivery
ICLA-Governed	ICLA-Core plus ICLA-8 and ICLA-9	Regulated or high-accountability operational capabilities
ICLA-Evolving	ICLA-Governed plus ICLA-11	Longitudinal institutional learning and evidence-based capability discovery

3. Add rule-based resolution and admission with explicit inadmissible outcomes; derive assemblies that bind knowledge and metric definitions and materialize them for at least two heterogeneous CEEs, such as an agent runtime and a human-review environment.
4. Record the situated CEE boundary, configuration, and execution identifier; submit evidence and validate contract-defined measurements; add impact analysis and governance; and implement successor construction, decision-linked deltas, append, and separately auditable activation.
5. Add pattern detection and capability-discovery workflows when sufficient history exists.

9.2 Companion Reference Specification and Repository

The companion artifact set comprises the versioned specification, linked constructed traces, deterministic reference implementation, and executable tests under `icla-spec 0.1.0`. It makes conformance inspectable and replayable within the exercised scope; the paper defines the architecture, invariants, and evaluation claims. The traces support a bounded ICLA-Evolving implementation-conformance claim over their combined scope: OAuth exercises the ICLA-Governed base and ICLA-11 pre-institutional boundary; formation exercises ICLA-11 from submitted proposal through identity assignment, CKC v1 append, retained lineage, a formed state without an active pointer, and separate activation. Automatic discovery, proposal quality, institutional judgment, and organizational effectiveness remain subjects of organizational evaluation.

The conformance coverage matrix (`specification/conformance-matrix.md`) maps each invariant to machine-checkable clauses, supporting schemas and traces, executable tests, and any governed-judgment remainder, while adding no architectural requirements.

The artifact set is organized as follows:

- `specification/schemas/`: machine-checkable contracts for the nine implemented object types;
- `specification/reference-objects/`: the consumer-independent object catalog and its schema mappings;
- `specification/reference-traces/oauth-042/`: linked execution, evidence, decision, succession, and activation records;
- `specification/reference-traces/auth-evolution-formation/`: submitted proposal, authorized formation decision, complete initial CKC v1, and before, formed, and active Registry states;
- `reference-implementation/`: deterministic resolution, validation, evidence qualification, adjudication, succession, formation, activation, and lineage preservation;
- `reference-implementation/tests/`: executable checks for both traces, conformance profiles, negative cases, historical preservation, and connected lineage.

Sect. 10 reports both traces. The metamodel and invariants assess structural consistency, schemas and linked records assess representational consistency, and deterministic tests assess executable consistency along both paths. Formation starts from a submitted proposal supported by retained-history references; discovery effectiveness and institutional judgment require organizational assessment. Sect. 11 relates this evidence to the longitudinal evaluation required by Claims C1–C3.

Companion artifact location removed for double-blind review.

10 Worked Example: Governed Agentic Software Engineering

This section reports the constructed design-science demonstration introduced in Sect. 3; organizational deployment remains prospective. The OAuth trace serializes the Registry snapshot, intent, resolution and admission, assembly, evidence, decision, and activation; deterministic fixtures exercise normal and negative paths. The companion executes resolution, conformance, lineage, and governed transitions around a represented CEE of coding-agent, validation, and human-review roles; internal work is scenario-instantiated, not benchmarked. Records attribute materializations, local state, outputs, and measurements; versioned fixture references represent Organizational Memory requirements, decisions, policies, client inventory, code, tests, release procedures, incidents, and prior evidence.

Here, API denotes application programming interface. The relevant Registry entries are `CAP-ARCH` (Architecture Governance), `CAP-IAM` (Identity and Access Architecture), `CAP-API` (API Evolution), `CAP-CHANGE` (Governed Code Change), `CAP-VERIFY` (Security and Compatibility Validation), and `CAP-RELEASE` (Controlled Release).

Before the intent arrives, an identity-policy change affects the source's authorized interpretation and temporal applicability. The event opens impact analysis over `CAP-IAM` and its dependency relations; governance then approves `CKC-IAM v8` and activates it through `ACT-IAM-008`, while earlier assemblies remain linked to v7.

CEE `CEE-OAUTH-042` generates intent `INT-OAUTH-042`: *Introduce the OAuth 2.1 authorization protocol across a multi-service platform while preserving backward compatibility for approved clients, regulatory controls, and auditable release evidence.*

10.1 Registry resolution and admissibility

Resolution `RES-OAUTH-042` runs against Registry snapshot `REG-SNAP-042`. Tab. 8 records candidate generation, mandatory `depends_on` expansion, exclusions, and admission. `CAP-DATA` adds no relevant obligation, while existing validation and release contracts already cover the required telemetry from `CAP-OBS`. Authorization, freshness, assurance, and budget checks pass. The stricter compatibility rule resolves the only conflict, so `ADM-OAUTH-042` admits six capabilities.

10.2 Assembly and materialization

The resolution snapshots six exact CKC versions: `ARCH v5`, `IAM v8`, `API v6`, `CHANGE v11`, `VERIFY v9`, and `RELEASE v4`. Assembly `ASM-OAUTH-042` records the contextual selection of affected services, approved clients, contracts, code, tests, policies, incidents, and rollout procedures. It excludes credentials, unrelated data, superseded policies, and unaffected services.

The assembly combines semantic policy commitments, procedural migration and validation guidance, and episodic incident and evidence knowledge; CKCs govern distributed-source bindings. It binds metrics and declares two substrate-specific materialization records—agent workspace `MAT-AGENT-042` and human-review packet `MAT-REVIEW-042`—with shared assembly semantics and lineage but separate represented local state. Fig. 6 shows the path.

10.3 Execution, evidence, and adjudication

Within the represented execution, both materializations are attributed to the composite CEE's

Table 8: Concrete Registry-resolution and admissibility trace for `INT-OAUTH-042`.

Stage	*Concrete result*	*Persistent record*
Candidate generation	Direct: `CAP-IAM`, `CAP-API`, `CAP-CHANGE`, `CAP-VERIFY`; lower-ranked: `CAP-DATA`, `CAP-OBS`	`RES-OAUTH-042` with scores and rationale
Relation expansion	Add `CAP-ARCH` and `CAP-RELEASE` through mandatory `depends_on` edges	Registry snapshot and traversed edges
Filtering and conflict handling	Exclude data and observability candidates; apply authorization, freshness, assurance, and stricter compatibility rule	Conformance trace with exclusions and policy versions
Final admission	Six capabilities retained; all mandatory assembly conditions satisfied	`ADM-OAUTH-042`: *admitted*

local state. Agent roles may index, coordinate, invoke tools, and iterate; reviewer roles work independently without continuous Registry, CKC, or assembly-service interaction. Only terminal or contract-defined checkpoint outputs, measurements, decisions, and provenance enter `EVD-OAUTH-042`. The bundle links them to the CEE, intent, Registry snapshot, admitted capabilities, assembly, exact CKC and source versions, materializations, and collection conditions. The episodic record may remain precedent or support a procedural commitment in successor `CKC-VERIFY v10`.

Tab. 9 reports the fixture measurements. All five governed metrics conform and pass; `agent.confidence` remains non-standard context and is excluded from institutional comparison because no selected CKC defines it. This negative case prevents an execution-local score from becoming institutional evidence or influencing activation.

Receipt `RCPT-OAUTH-042` qualifies the evidence for review. Decision `DEC-OAUTH-042` accepts the conforming artifacts and measurements. Its successor delta records the addition of a reusable compatibility-test pattern to complete successor `CKC-VERIFY v10`; the service-local migration exception remains local evidence. `ACT-VERIFY-010` activates v10 only for future resolutions; `ASM-OAUTH-042` remains linked to v9.

10.4 CKC succession and capability crystallization

The trace keeps the two evolution paths distinct. `DEC-OAUTH-042` exercises succession by deepening `CAP-VERIFY`'s CKC lineage because the test pattern remains within its existing responsibility rather than defining a separate capability.

The single OAuth trace provides insufficient recurrence evidence, so `PROP-AUTH-EVOL-01` remains in candidate state. Later executions involving token rotation, provider replacement, scope redesign, client migration, and protocol upgrades indicate the evidence required before submission and institutional review. The candidate is distinct from a successor to `CAP-IAM` or `CAP-VERIFY` because its outcome would be coordinated authorization-mechanism evolution across providers, clients, scopes, and protocols. Its record includes a cross-intent assembly signature, comparable outcomes, candidate owner, overlap analysis, and draft CKC. Review may retain it as a template or extend the Registry through governed promotion. Earlier traces remain unchanged.

The separate `auth-evolution-formation` trace exercises formation from the same submitted responsibility supported by additional retained authentication-change references. An authorized decision assigns `CAP-AUTH-EVOL` and appends complete `CKC-AUTH-EVOL v1`; the resulting formed state has no active pointer, and

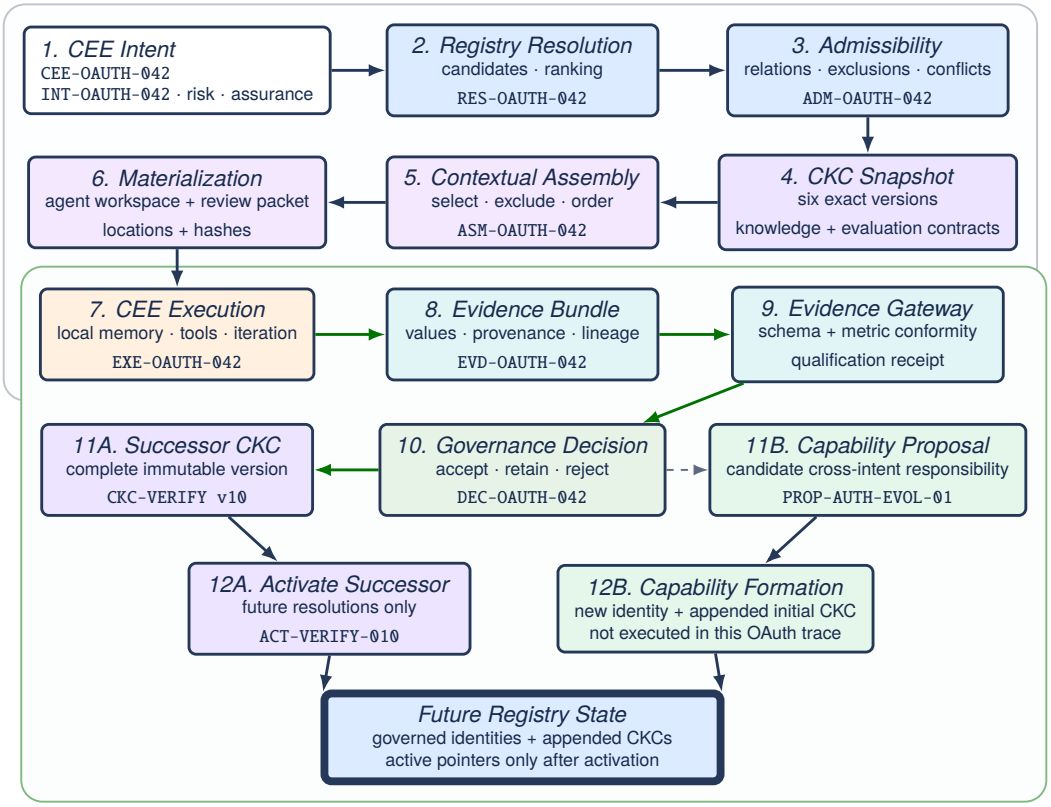


Figure 6: Constructed auditable CEE trace. Records represent exact CKC-derived materializations delivered to an autonomous CEE interval; selected evidence returns for qualification and adjudication while earlier traces remain immutable. The OAuth crystallization branch stops at candidate because one execution provides insufficient recurrence evidence; a separate submitted-proposal trace exercises governed formation.

a separate activation publishes v1. Tests cover authority, identity uniqueness, CKC completeness and provenance, lineage, activation separation, and negative cases. The supporting histories serve as checked provenance references; longitudinal evidence and the institutional adequacy of recurrence, boundary, ownership, overlap, value, and promotion judgments remain subjects of organizational evaluation.

The domain is software engineering; the architecture is domain-independent.

11 Artifact Evaluation and Comparative Protocol

This section separates current artifact evidence from prospective organizational claims. It first assesses structural, representational, and executable consistency within the implemented scope, then states the falsifiable claims, comparative conditions, and measurement levels required for independent evaluation.

11.1 Present artificial artifact evaluation

The summative artificial evaluation assesses fidelity, declared-scope coverage, consistency, traceability, reproducibility, and technical instantiation. Structural consistency concerns constructs, relations, and invariants; representational consistency concerns schema-conformant linked records; and executable consistency concerns implemented invariants during deterministic replay. The prospective comparative protocol addresses organizational performance.

Following Prat et al. (2015), the evidence addresses artifact structure, while organizational fit

Table 9: Governed measurement fixture in evidence bundle `EVD-OAUTH-042`.

Metric identifier	*Reported value*	*Governed threshold*	*Gateway result*
`iam.profile.coverage`	100% endpoints	= 100%	Conforming; pass
`api.client.compatibility`	48/48 clients	= 100% approved clients	Conforming; pass
`verify.high.findings`	0 findings	= 0	Conforming; pass
`release.rollback.time`	4.2 minutes	≤ 5 minutes	Conforming; pass
`verify.validation.latency`	34 minutes	≤ 45 minutes	Conforming; pass
`agent.confidence`	0.91 score	No governed definition	Non-standard; excluded

remains prospective. Tab. 10 summarizes the evidence and boundaries. The exercised succession and submitted-proposal formation paths receive structural, representational, and deterministic assessment. Formation verifies identity assignment, complete CKC v1 append, provenance and lineage, a formed state without an active pointer, and separate activation. These results support internal coherence and bounded technical feasibility: modeled-path executability and invariant preservation, not throughput, production integration, distributed consistency, or organizational-scale operation. Claims C1–C3 frame organizational evaluation of discovery, judgment quality, outcomes, and governance cost.

11.2 Falsifiable comparative claims

The following protocol states three falsifiable claims for comparative evaluation. In the ICLA condition, capability-level measurements use CKC-defined metrics, units, collection conditions, and thresholds, supporting comparison of outcomes across CEEs and time under the applicable CKCs' declared conditions. Sample sizes, directional hypotheses, non-inferiority margins, and acceptance thresholds are preregistered.

Claim C1—cross-CEE canonical stability. The unit is one recurring intent executed by heterogeneous CEEs—for example, an agent runtime, deterministic workflow, and human team—from the same governed sources and active CKCs. Measures are canonical object count, divergent authoritative copies, synchronization actions, CEE-specific exceptions, and task quality. C1 is supported when ICLA reduces divergence and synchronization against both baselines without violating declared quality or assurance margins.

Claim C2—continuous change accountability. The unit is one injected source, policy, obligation, relation, owner, or lifecycle change. Measures are affected-capability recall and precision, owner-identification time, review and activation latency, stale pointers, unexplained assembly changes, and reproducibility. C2 is supported by higher impact accuracy, fewer stale or unexplained states, and complete decision and activation traces.

Claim C3—controlled institutional discovery. The unit is a retained-history pattern independently classified under preregistered criteria as promotable, incidental, or non-distinct. Measures are proposal precision and recall, false promotion, reviewer agreement, owner-assignment time, duplication, rollback, and post-promotion value. False promotion means promoting an incidental or non-distinct case. C3 is supported when limits are met and promotions remain distinct from CKC succession and traceable to review evidence.

11.3 Comparative conditions

A longitudinal study should exercise the same recurring capability scenario under three conditions:

Table 10: Analytical evaluation of the ICLA artifact and evidentiary boundary.

Criterion	*Evaluation evidence*	*Assessment and boundary*
Fidelity and relevance	Architectural gap positioned against prior work; requirement-to-component traceability	Represents the identified continuity and authority problem; the synthesis is integrative rather than exhaustive
Declared-scope coverage	Information objects, transitions, logical interfaces, and ICLA-1–ICLA-11	Covers the declared responsibilities and failure states; domain-specific obligations remain outside the reference model
Internal consistency	Formal relations, immutable-version rules, schemas, and conformance tests	Separates stable identity, active authority, projection, execution, evidence, and activation across the reference lifecycle
Traceability and reproducibility	Linked OAuth 2.1 and capability-formation records with exact CKC, source, policy, proposal, decision, formation, and activation references	Supports explanation and replay under access and retention policy; history beyond OAuth is referenced provenance, while longitudinal evidence requires organizational collection
Technical instantiation	CEE materialization records, governed fixture measurements, non-standard-metric rejection, successor append and activation, submitted-proposal formation, CKC v1 append, and separate initial activation	Executes governed succession and formation; source retrieval, internal CEE work, automatic discovery, distributed operation, and promotion adequacy require separate assessment

1. *Retrieval baseline*: artifact retrieval without persistent capability identity or lineage authority;
2. *Context-orchestration baseline*: governed context delivery without capability-level succession or identity formation;
3. *ICLA condition*: Registry-addressable institutional capabilities, versioned CKCs, derived assemblies, governed evolution, and capability crystallization.

These labels denote design conditions, not claims that every retrieval or context-orchestration system lacks governance. Each baseline should be implemented as a credible contemporary alternative, matched on sources, access controls, CEE configurations, and budget. Classification must follow the mechanisms present, not product names. A baseline that implements persistent capability identity, versioned contracts, and governed succession has converged on part of the ICLA condition and must be reported accordingly. The intended contrast concerns the presence and scope of explicit capability-level identity, authority, and lineage mechanisms—not differences in retrieval quality, access control, or executor configuration.

The study includes at least two heterogeneous CEEs, repeated intents, source and policy changes, rejected evidence, a successor CKC, and a recurrent assembly considered for discovery. Conditions share source snapshots and intent sequence; model and tool versions are pinned and condition order counterbalanced. Where feasible, reviewers are blind to labels. Results report uncertainty and governance effort alongside task outcomes.

11.4 Measurement levels

Execution measures include task success, time, error, escalation, cost, and artifact quality. Internal measures cover Registry coverage, resolution accuracy, CKC scope, assembly relevance, policy compliance, provenance, and conflict detection. Institutional measures cover maintenance effort, infrastructure and computation cost, evidence disposition, change-to-adjudication latency, divergence, stale-knowledge half-life, rollback, and Registry growth. Capability-level metric definitions and collection conditions remain versioned in the applicable CKC evaluation contracts; study-level measures are specified separately by the

comparative protocol. Analysis separates boundary failures, projection failures, CEE execution failures, evidence-quality failures, and governance or evolution failures so that downstream performance is attributed to the responsible layer.

12 Discussion

This discussion relates ICLA's conceptual contribution to its evidentiary and adoption boundaries.

12.1 Contribution to conceptual modeling and enterprise architecture

ICLA contributes a governed relational structure connecting capability identity, versioned contracts, contextual execution, evidence, and authorized evolution. Enterprise and capability models already represent goals, KPIs, context, realization, observation, decision, and adjustment; runtime models retain operational representations; and model management preserves heterogeneous relations and evolution. ICLA organizes these established concerns around the institutional capability lineage.

The contribution depends on four linked separations: capability identity from authorized CKC versions; a CKC from its execution-specific assembly; governed commitments from CEE-local execution logic or reasoning; and submitted evidence from governed canonical change. Coupling identity to the current contract erases historical interpretation; coupling contract and projection turns situated context into canonical state; coupling governed commitments and CEE-local execution logic or reasoning allows executor-local state to redefine the institutional mandate; and coupling execution output and institutional change delegates promotion authority to the executor. Conflating succession with crystallization further obscures whether an institution refined an existing responsibility or recognized a new one. The lineage makes these failure modes jointly testable through typed links, invariants, and transition records.

The contribution is conceptual-modeling because it specifies implementation-independent constructs, typed relations, cardinalities, lifecycle states, and transitions; separates authority and execution levels; and defines conformance conditions for comparing realizations. The companion provides a machine-checkable and replayable realization of that kernel within the exercised scope without defining its meaning.

Relative to Sect. 2, ICLA makes explicit the governed relations among an institutional capability identity, its knowledge-and-evaluation contracts, a contract-governed evidence path, and activation records that preserve earlier references without tracking every internal execution state. It also governs whether a recurrent responsibility warrants identity, ownership, lifecycle, and an initial CKC. Organizational benefit remains to be tested.

Human–AI execution makes one boundary explicit: operational autonomy does not authorize institutional redefinition. Configurations may perform the capability and produce candidate contributions while the lineage records accountable change.

12.2 Practical implications and adoption boundary

ICLA complements rather than replaces RAG, agent memory, or context-orchestration mechanisms, which may remain inside the assembly and CEE layers. Adoption can target only capabilities whose continuity has sufficient institutional value.

Architectural fit depends on whether continuity and accountability justify lifecycle overhead. Benefits include avoided losses from stale, divergent, or untraceable execution, plus reuse, impact analysis, reproducibility, and accountable change; they increase with recurrence, continuity horizon, CEE and source heterogeneity, evidence requirements, and consequence of error.

Costs span bootstrap and operation. Bootstrap establishes initial capabilities, owners, CKCs, and source bindings before normal resolution and evolution; ongoing costs cover maintenance, infrastructure, qualification, review latency, and organizational change. Governance latency is concentrated at admission or re-resolution, qualification, adjudication, and activation—not within CEE execution. Calibration remains future work.

Transient, low-risk, or weakly recurrent work should normally remain outside ICLA. Adoption can also be proportional. ICLA-Core supports identity, projection, traceability, and reproducibility; ICLA-Governed adds evidence-based canonical change for high-accountability capabilities; and ICLA-Evolving defines the governed conformance boundary for capability discovery. Deployments may operationalize institutional discovery when sufficient history and expected value justify its additional cost. A pilot should therefore begin with a small set of recurrent, high-impact capabilities and reuse existing repositories, workflows, catalogs, and agent runtimes.

12.3 Limitations and threats to validity

This paper defines a reference architecture, conceptual meta-model, conformance model, and constructed traces. The meta-model and invariants assess structural consistency; companion records and schemas assess representational consistency; and tests assess executable consistency through successor activation and bounded formation. These results establish artifact coherence and bounded technical feasibility for implemented succession and submitted-proposal formation paths. Independent organizational studies must evaluate discovery effectiveness, judgments about recurrence, boundary, ownership, overlap, value, and promotion, and comparative effectiveness.

The same author designed the artifact, scenarios, implementation, and tests. Negative cases and explicit invariant checks strengthen internal assessment; Claims C1–C3 require independent replication and comparative deployment.

The integrative synthesis supports construct selection, boundary refinement, and positioning; exhaustive coverage and uniqueness claims require a systematic review. Recent AI preprints characterize the execution context. Empirical research should relate the new terms to established capability-model concepts. External validity currently rests on constructed software-engineering traces and a logically unified Registry. Capability boundaries depend on purpose, ownership, risk, and language; evolution depends on observable events, reliable bindings, and timely adjudication; and recurrence may create governance noise rather than value.

12.4 Future research

Claims C1–C3 define a longitudinal agenda across heterogeneous CEEs, repeated intents, controlled changes, proposal quality, and governance effort. Research should evaluate pattern detection, proposal generation, capability-boundary methods, inter-reviewer agreement, centralized and federated Registries, cross-organization exchange, authority conflicts, privacy-preserving evidence, and capability equivalence. Field studies should identify when risk and recurrence justify the additional governance layer.

13 Conclusion

ICLA defines institutional capability lineage as the governed structure connecting stable capability identity, versioned contracts, situated execution, evidence, and controlled change across distributed sources. Registry pointers select authority; CKCs define authorized commitments; and assemblies preserve exact references to the CKC and source versions used.

ICLA delegates execution, not institutional authority. CEEs control local work, while contract-selected evidence enters adjudication before canonical change. Succession deepens an existing CKC lineage; crystallization adds a new capability identity and initial CKC; both preserve retained history.

The companion makes these boundaries inspectable and replayable, establishing executable consistency for succession and submitted-proposal formation with separate activation. Discovery effectiveness, promotion judgment, and organizational effectiveness remain prospective. ICLA targets recurrent, consequential capabilities whose benefits exceed lifecycle cost.

A Logical Interfaces and Implementation Mapping

Tab. 11 defines protocol-independent logical operations; Tab. 12 provides one practical candidate implementation mapping.

Table 11: Logical interfaces. Retention expiry and authorized disposal remain storage-lifecycle concerns outside this operation set under the conditions accompanying Eq. (18).

Operation	*Input*	*Output*	*Purpose*
`resolve_intent`	Intent i_η with semantic content and situated conditions	Ranked candidates/exclusions; admitted set and exact CKC map; Registry/matcher versions; rationale, confidence, status	Resolve a situated need and freeze the authorized contract selection before retrieving content
`get_capability`	Capability identifier, version policy	Metadata, lifecycle, active CKC, latest governed CKC, lineage, relations	Navigate and distinguish identity, current authority, and governed succession
`assemble`	Admitted resolution with exact CKC map and Registry snapshot, policies	Assembly plan, immutable trace, evaluation contracts	Select knowledge and bind governed measures without changing the admitted contract selection
`materialize`	Assembly identifier, CEE substrate	Bundle, payload, or access handles; evaluation and evidence-contract metadata; hash; expiry	Create a CEE-consumable representation while preserving assembly semantics, traceability, and evaluation bindings
`submit_evidence`	CEE reference, assembly identifier, evidence report	Receipt, conformance, qualification status	Validate evidence conformity and contract-defined measurements, preserve lineage, and begin governance
`adjudicate`	Maintenance-event references, policies, reviewers	Decision record and authorized actions	Record compatible dispositions and actions without mutating canonical state
`append_successor`	Capability, predecessor CKC, complete successor CKC, decision-linked delta, authorizing decision	Append receipt, updated CKC lineage, inactive-successor status	Persist the complete successor and its explanatory transition without activating it
`propose_capability`	Candidate responsibility, supporting retained-history references, formation rationale, owner candidate, draft CKC	Proposal identifier and state	Create a pre-institutional proposal without identity
`promote_capability`	Submitted proposal, Registry snapshot, complete CKC v1, formation decision	New identity, formation append, formed snapshot	Assign identity and append CKC v1 without activation
`activate_ckc`	Capability, appended CKC, successor or formation append, activation decision	Active-pointer update and propagation record	Publish a successor or initial CKC for future resolutions
`impact_analysis`	Identified maintenance event affecting a source, evidence, policy, ownership or lifecycle state, CKC, relation, or capability	Affected source bindings, capabilities, exact CKCs, relation paths, retained assemblies, CEEs or consumers, and review case	Maintain freshness and identify governed work

Table 12: One practical implementation mapping for ICLA.

Logical concern	*Candidate implementation*	*Stored information*
Registry and CKC management	Relational or graph metadata plus versioned object storage	Identity, ownership, relations, lifecycle, immutable CKCs, lineages, active pointers
Organizational Memory	Document, code, data, and knowledge repositories with change feeds	Versioned sources, bindings, retention, change events
Resolution and assembly	Rules, graph traversal, hybrid search, optional model assistance	Ranked capabilities, assembly plan, evaluation bindings, conformance trace
Materialization and delivery	Workspace builder, serializer, representational state transfer (REST), Model Context Protocol (MCP), messaging, or files	Bundle, evaluation schema, uniform resource identifier (URI), hash, expiry, execution handle
Distributed capability execution	Human workbench, agent runtime, workflow, software service, or hybrid	CEE boundary, relevant configuration, and submitted outputs; local state remains under CEE authority within the admitted mandate
Evidence Gateway	Schema, provenance, and contract-defined measurement validation with append-only storage	Results, artifacts, measurements, provenance, signatures
Governance and evolution	Review workflow, policy engine, audit log, dependency graph	Impact cases, decisions, successor deltas and appends, formation appends, activations, rollback, proposals
Explorer interface	Registry APIs and graph queries	Search, navigation, versions, evidence, candidates

References

Alavi M., Leidner D. E. (2001) Review: Knowledge Management and Knowledge Management Systems: Conceptual Foundations and Research Issues. In: MIS Quarterly 25(1), pp. 107–136

Augusto A., Conforti R., Dumas M., La Rosa M., Maggi F. M., Marrella A., Mecella M., Soo A. (2019) Automated Discovery of Process Models from Event Logs: Review and Benchmark. In: IEEE Transactions on Knowledge and Data Engineering 31(4), pp. 686–705

Blair G., Bencomo N., France R. B. (2009) Models@run.time. In: Computer 42(10), pp. 22–27

Bradner S. (1997) Key Words for Use in RFCs to Indicate Requirement Levels. RFC 2119

Fill H.-G. (2018) Semantic Annotations of Enterprise Models for Supporting the Evolution of Model-Driven Organizations. In: Enterprise Modelling and Information Systems Architectures 13(5), pp. 1–25

Foss N. J., Husted K., Michailova S. (2010) Governing Knowledge Sharing in Organizations: Levels of Analysis, Governance Mechanisms, and Research Directions. In: Journal of Management Studies 47(3), pp. 455–482

Frank U. (2014) Multi-Perspective Enterprise Modeling: Foundational Concepts, Prospects and Future Research Challenges. In: Software and Systems Modeling 13(3), pp. 941–962

Gao H., Lulla J. L., Lin H. Y., Baltes S., Treude C., Zahedi M. (2026) From Registry to Repository: How AI Agent Skills Are Written, Adapted, and Maintained. In: arXiv preprint arXiv:2607.00911

Garralda-Barrio M. (2026) Knowledge-Centric Information Systems. In: arXiv preprint arXiv:2607.02609

Gau M., Maedche A., vom Brocke J. (2023) Towards an Open Repository for Design Science Research: A Meta-Model and its Instantiation for the Representation of Design Science Research Processes. In: Enterprise Modelling and Information Systems Architectures 18(4), pp. 1–22

Grant R. M. (1996) Toward a Knowledge-Based Theory of the Firm. In: Strategic Management Journal 17(S2), pp. 109–122

Hartmann T., Fouquet F., Nain G., Morin B., Klein J., Barais O., Le Traon Y. (2014) A Native Versioning Concept to Support Historized Models at Runtime. In: Model-Driven Engineering Languages and Systems. Lecture Notes in Computer Science Vol. 8767. Springer, pp. 252–268

Hernandez-Mendez A., Michel F., Matthes F. (2018) A Practice-Proven Reference Architecture for Model-Based Collaborative Information Systems. In: Enterprise Modelling and Information Systems Architectures, pp. 262–272

Hevner A. R., March S. T., Park J., Ram S. (2004) Design Science in Information Systems Research. In: MIS Quarterly 28(1), pp. 75–105

ISO/IEC/IEEE (2022) ISO/IEC/IEEE 42010:2022: Software, Systems and Enterprise—Architecture Description. International Organization for Standardization

Kang J., Ji M., Zhao Z., Bai T. (2025) Memory OS of AI Agent. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. ACL, pp. 25961–25970

Koutsopoulos G., Andersson A., Stirna J., Henkel M. (2024) Application and Evaluation of Interlinked Approaches for Modeling Changing Capabilities. In: Software and Systems Modeling 23(4), pp. 895–924

Koutsopoulos G., Henkel M., Stirna J. (2021) An Analysis of Capability Meta-Models for Expressing Dynamic Business Transformation. In: Software and Systems Modeling 20, pp. 147–174

Lebo T., Sahoo S., McGuinness D. (Apr. 2013) PROV-O: The PROV Ontology. World Wide Web Consortium, W3C Recommendation

Lewis P., Perez E., Piktus A., Petroni F., Karpukhin V., Goyal N., Küttler H., Lewis M., Yih W.-t., Rocktäschel T., Riedel S., Kiela D. (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems Vol. 33, pp. 9459–9474

Li Z., Song S., Xi C., Wang H., Tang C., Niu S., Chen D., Yang J., Li C., Yu Q., Zhao J., Wang Y., Liu P., Lin Z., Wang P., Huo J., Chen T., Chen K., Li K., Tao Z., Lai H., Wu H., Tang B., Wang Z., Fan Z., Zhang N., Zhang L., Yan J., Yang M., Xu T., Xu W., Chen H., Wang H., Yang H., Zhang W., Xu Z.-Q. J., Chen S., Xiong F. (2025) MemOS: A Memory OS for AI System. In: arXiv preprint arXiv:2507.03724

Mei L., Yao J., Ge Y., Wang Y., Bi B., Cai Y., Liu J., Li M., Li Z.-Z., Zhang D., Zhou C., Mao J., Xia T., Guo J., Liu S. (2025) A Survey of Context Engineering for Large Language Models. In: arXiv preprint arXiv:2507.13334

Nonaka I., Takeuchi H. (1995) The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation. Oxford University Press

Peffers K., Tuunanen T., Rothenberger M. A., Chatterjee S. (2007) A Design Science Research Methodology for Information Systems Research. In: Journal of Management Information Systems 24(3), pp. 45–77

Prat N., Comyn-Wattiau I., Akoka J. (2015) A Taxonomy of Evaluation Methods for Information Systems Artifacts. In: Journal of Management Information Systems 32(3), pp. 229–267

Qin X., Luan S., See J., Boukhers Z., Yang C., Li Z. (2026a) Governed Capability Evolution: Lifecycle-Time Compatibility Checking and Rollback for AI-Component-Based Systems, with Embodied Agents as Case Study. In: arXiv preprint arXiv:2604.08059

Qin X., Luan S., See J., Yang C., Li Z. (2026b) Learning Without Losing Identity: Capability Evolution for Embodied Agents. In: arXiv preprint arXiv:2604.07799

Redlich D., Blair G., Rashid A., Molka T., Gilani W. (2014) Research Challenges for Business Process Models at Run-Time. In: Bencomo N., France R. B., Cheng B. H. C., Aßmann U. (eds.) Models@run.time: Foundations, Applications, and Roadmaps. Lecture Notes in Computer Science Vol. 8378. Springer, pp. 208–236

Salay R., Kokaly S., Di Sandro A., Fung N. L. S., Chechik M. (2020) Heterogeneous Megamodel Management Using Collection Operators. In: Software and Systems Modeling 19, pp. 231–260

Squire L. R. (1992) Declarative and Nondeclarative Memory: Multiple Brain Systems Supporting Learning and Memory. In: Journal of Cognitive Neuroscience 4(3), pp. 232–243

Stein E. W. (1995) Organization Memory: Review of Concepts and Recommendations for Management. In: International Journal of Information Management 15(1), pp. 17–32

Stirna J., Grabis J., Henkel M., Zdravkovic J. (2012) Capability Driven Development—An Approach to Support Evolving Organizations. In: The Practice of Enterprise Modeling. Lecture Notes in Business Information Processing Vol. 134. Springer, pp. 117–131

Stojanovic L., Stojanovic N., Maedche A. (2003) Change Discovery in Ontology-Based Knowledge Management Systems. In: Advanced Conceptual Modeling Techniques. Lecture Notes in Computer Science Vol. 2784. Springer, pp. 51–62

Subramaniam P., Ma Y., Li C., Mohanty I., Castro Fernandez R. (2021) Comprehensive and Comprehensible Data Catalogs: The What, Who, Where, When, Why, and How of Metadata Management. In: arXiv preprint arXiv:2103.07532

Sun L., Yang Y., Duan Q., Shi Y., Lyu C., Chang Y.-C., Lin C.-T., Shen Y. (2025) Multi-Agent Coordination across Diverse Applications: A Survey. In: arXiv preprint arXiv:2502.14743

Tabassi E. (2023) Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. National Institute of Standards and Technology

Tulving E. (1972) Episodic and Semantic Memory. In: Tulving E., Donaldson W. (eds.) Organization of Memory. Academic Press, New York, pp. 381–403

van der Aalst W. M. P., Weijters A. J. M. M., Maruster L. (2004) Workflow Mining: Discovering Process Models from Event Logs. In: IEEE Transactions on Knowledge and Data Engineering 16(9), pp. 1128–1142

Venable J., Pries-Heje J., Baskerville R. (2016) FEDS: A Framework for Evaluation in Design Science Research. In: European Journal of Information Systems 25(1), pp. 77–89

Walsh J. P., Ungson G. R. (1991) Organizational Memory. In: Academy of Management Review 16(1), pp. 57–91

Wu Q., Bansal G., Zhang J., Wu Y., Li B., Zhu E., Jiang L., Zhang X., Zhang S., Liu J., Awadallah A. H., White R. W., Burger D., Wang C. (2024) AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In: First Conference on Language Modeling

Yang Y., Chai H., Shao S., Song Y., Qi S., Rui R., Zhang W. (2025) AgentNet: Decentralized Evolutionary Coordination for LLM-Based Multi-Agent Systems. In: Advances in Neural Information Processing Systems Vol. 38

Zdravkovic J., Stirna J., Henkel M., Grabis J. (2013) Modeling Business Capabilities and Context Dependent Delivery by Cloud Services. In: Advanced Information Systems Engineering. Lecture Notes in Computer Science Vol. 7908. Springer, pp. 369–383

Zhang Q., Hu C., Upasani S., Ma B., Hong F., Kamanuru V., Rainton J., Wu C., Ji M., Li H., Thakker U., Zou J., Olukotun K. (2026) Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. In: International Conference on Learning Representations

Zhang Z., Dai Q., Bo X., Ma C., Li R., Chen X., Zhu J., Dong Z., Wen J.-R. (2025) A Survey on the Memory Mechanism of Large Language Model-Based Agents. In: ACM Transactions on Information Systems 43(6), pp. 1–47

Zou H. P., Huang W.-C., Wu Y., Guo J., Chen Y., Miao C., Nguyen H. H., Zhou Y., Zhang W., Fang L., Zhang H., Wang F., Zhang P., He L., Li Y., Li D., Jiang R., Yu P. S. (2026) LLM-Based Human-Agent Collaboration and Interaction Systems: A Survey. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, pp. 36335–36364